

EXTp: A Soundness Theorem for Counterfactual Exploit Replay on Formally Verified Microkernels

Utku Erol

Independent Researcher

utkuerol71@gmail.com

Abstract—Counterfactual exploit analysis — determining which input parameter causally produced an observed outcome — is pervasive in security research, but its current practice is epistemically unsound. Three gaps separate “different outcome” from “caused by the change”: non-deterministic replay noise, replay-machinery side-effects, and causal pathways traversing microarchitectural state invisible to the analyst’s surface. We present EXTp, a counterfactual replay framework on bare-metal seL4, and the Counterfactual Soundness Theorem (CST) — the first formal soundness theorem for counterfactual causal attribution grounded in a verified microkernel substrate. EXTp realizes three observable properties: **Capability Separation** (Pearl modularity; Clopper-Pearson bound $\leq 0.37\%$ at $N=998$), **Strong Observational Equivalence** (replay determinism: $\text{score}_S=1.0000$ observed over $N=34$ cross-boot trials, Clopper-Pearson one-sided 95% upper bound on the per-run failure rate $\leq 10.4\%$), and **Measurement Interference** (calibrated timing threshold, realized false-positive rate 0.61%–2.78%). CST proves their composition grounds Pearl/Halpern-Pearl causal attribution within enumerated epistemic bounds; CST v1.0 establishes soundness under single-VCPU intervention scope and bare-metal microcode-stable configurations, with multi-VCPU and cross-configuration extensions charted as future work. A synthetic analog modeled on the CrackArmor confused-deputy pattern, executed on bare-metal Intel Alder Lake, exercises the pipeline end-to-end and shows the framework correctly detecting its own A1-modulo violations.

I. INTRODUCTION

Determining *why* an exploit succeeds is harder than determining *that* it succeeds. A crash, a privilege escalation, a sandbox escape — these are observable. Whether a specific input parameter *caused* the outcome, or whether a different parameter value would have prevented it, is a counterfactual question. Current practice answers it informally: re-run with a modified input, observe whether the outcome changes, call it causal. This methodology is pervasive, and it is epistemically unsound. Three gaps separate “different outcome” from “caused by the change”: non-deterministic replay noise that can mimic causal signals; replay machinery side-effects that confound the measurement; and causal pathways that traverse microarchitectural state invisible to the analyst’s observable surface. Without formal guarantees addressing all three, counterfactual exploit analysis is intuition dressed as method.

The theoretical machinery to close these gaps exists. Pearl’s structural causal model [1] defines the $\text{do}(X=x')$ intervention operator and the axioms — modularity, faithfulness, effective-

ness — that make causal attribution provably sound. Halpern and Pearl’s actual-causation conditions [2] specify precisely when an observed outcome is attributable to a named cause. Yet this apparatus has not been applied to exploit analysis. The obstacle is structural: Pearl’s framework is probabilistic and abstract, defined over distributions and idealized structural equations. Exploit replay operates on concrete microarchitectural traces — TSC measurements with asymmetric noise floors, VM-exit surfaces with finite register coverage, capability-mediated VMM machinery with empirical rather than axiomatic side-effect bounds. To our knowledge, no prior work formally bridges this gap.

We present **EXTp**, and with it the **first soundness theorem for counterfactual causal attribution in exploit analysis grounded in a formally verified substrate**. Prior Pearl-applied work in adjacent domains [3] (AID, SIGMOD 2020) demonstrates probabilistic causal attribution for database failures but does not provide a soundness theorem grounded in a verified computational base; conceptual extensions of the HP framework [4] provide no operational system. CST is the first theorem-grade formalization of admissibility under enumerated antecedents on a verified substrate. EXTp is a counterfactual replay framework on a bare-metal seL4 [5] virtual machine monitor. Its Counterfactual Soundness Theorem (CST) proves that, under the formal assumptions grounded in EXTp’s observable properties, replay-based causal attribution satisfies Pearl’s effectiveness and composition axioms (modularity realized by CapSep, faithfulness subsumed by SOE) and Halpern-Pearl’s AC1 and AC2 actual-causation conditions — with divergence points (irreversibility inapplicable; determinism strictly stronger than faithfulness) declared explicitly rather than elided. The result is a system where “ $\text{do}(X=x')$ caused divergence D at event i ” is not an analyst’s intuition but a formally admissible claim within an enumerated epistemic envelope.

EXTp achieves this through three observable properties that compose into CST. **Capability Separation (CapSep)** proves intervention authority disjointness ($\mathcal{K}_{\text{int}} \cap \mathcal{K}_{\text{fwk}} = \emptyset$), realizing Pearl’s modularity axiom: $\text{do}(X=x')$ cannot directly modify framework-internal state. For seL4’s formally verified operations, this follows from seL4’s Isabelle/HOL proof harness; for extended VT-x operations, it is empirically bounded with Clopper-Pearson confidence. **Strong Observational Equivalence (SOE)** establishes trajectory-level replay determinism

(score_S = 1.0000, $N=34$ cross-boot trials), a property strictly stronger than Pearl’s probabilistic faithfulness and sufficient to rule out coincidence as an alternative explanation for observed divergence. **Measurement Interference (MI)** derives a calibrated timing detection threshold $\Delta^* = z_{1-\alpha} \cdot \hat{\sigma}_{\text{baseline}}$ with empirically realized false-positive rate 0.61%–2.78%, bounding the residual confounding that CapSep does not eliminate. CST proves that together these three properties are sufficient to ground Pearl-style causal attribution within an explicitly enumerated epistemic envelope.

This paper makes four contributions. **(C1)** EXTp: a counterfactual exploit replay framework on bare-metal seL4 with four intervention classes and cross-boot replay stability score_S = 1.0000 over $N=34$ trials. **(C2)** Three independently verifiable observable properties — CapSep (modularity axiom), SOE (faithfulness/determinism), MI (confounder boundedness) — each making falsifiable claims about a distinct dimension of the replay machinery. **(C3)** CST: the first formal soundness theorem for counterfactual causal attribution in exploit analysis, with explicit Pearl/Halpern-Pearl correspondence, declared divergence points, and a proof whose *conditional core* — the composition of the antecedent envelope (A_1, A_2, A_3, A_4) into the Pearl/Halpern-Pearl correspondence properties — is machine-checked in Isabelle/HOL, with the empirical antecedents calibration-supplied and the seL4 lifting left to future work (§VII-C). **(C4)** A privilege-check-bypass demonstration modeled on the CrackArmor confused-deputy exploit pattern¹ on bare-metal seL4 showing an end-to-end CST-admissible causal attribution from a forged capability token to an observable VM-exit divergence.

The remainder of this paper is structured as follows. §II provides background on seL4, VT-x VM-exits, and Pearl causality. §III defines the threat model. §IV describes EXTp’s framework design. §V states and proves the four formal properties. §VI presents the evaluation. §VII discusses scope and downstream applications. §VIII surveys related work. §IX concludes.

II. BACKGROUND

A. seL4 and the Capability Model

seL4 is a formally verified L4 microkernel whose functional correctness, integrity, and confidentiality properties are mechanically proven in Isabelle/HOL [5], [6]. EXTp uses seL4 as its VMM substrate, inheriting the proof as a trust anchor for Capability Separation. seL4’s capability model partitions authority into unforgeable tokens: a thread may only access a resource if it holds a capability to that resource, and capabilities are manipulated exclusively through kernel-mediated operations.

For EXTp’s purposes, two operation sets are distinguished. $\mathcal{K}_{\text{verified}} \supseteq \{\text{create, copy, revoke, destroy}\}$ comprises the capable operations that fall within seL4’s L4.verified formal

scope; the listed subset is those operations directly relevant to EXTp’s intervention surface, while $\mathcal{K}_{\text{verified}}$ in full is the larger L4.verified-covered API. Their external atomicity — the property that state transitions are observable as single steps from outside the kernel — is established by the Isabelle proof [5]. $\mathcal{K}_{\text{extended}} \supseteq \{\text{MapEPT, Unmap, WriteVMCS, ReadVMCS}\}$ (full enumeration in Appendix A) comprises the x86-64 VT-x extension operations that EXTp’s intervention machinery uses directly; these lie outside L4.verified’s formal scope and their atomicity properties are empirically bounded (Clopper-Pearson 95% CI $\leq 10.4\%$ at $N=34$ cross-boot; the CapSep-level bound $\leq 0.37\%$ at $N=998$ is detailed in §V-A). The distinction matters: CapSep’s formal guarantee for $\mathcal{K}_{\text{verified}}$ is unconditional; for $\mathcal{K}_{\text{extended}}$ it is empirically conditioned and subject to Q4 validation (§V-A).

B. VT-x VM-exits and the Observable Surface

Intel VT-x delivers control to the VMM on every VM-exit, populating a VMCS exit-information field with the exit reason, exit qualification, and guest register state. EXTp captures a 4-tuple observable surface per VM-exit:

$$\text{observable}(X) = \langle \text{rip}(X), \text{exit_reason}(X), \text{exit_qual}(X), \text{rax}(X) \rangle \quad (1)$$

This surface is intentionally narrow. Other general-purpose registers (rbx, rcx, ..., r15) are captured internally but excluded from the formal surface to maintain falsifiability — a surface defined over all registers would require claims about register-level determinism that EXTp’s current calibration does not support (§V-B). The surface is sufficient for the class of exploits EXTp targets: VM-exit-boundary privilege escalations whose causal signal manifests in instruction-pointer divergence, exit-reason changes, or return value modification.

C. Pearl Causality and Actual Causation

Pearl’s structural causal model formalizes causation through the $\text{do}(X=x')$ intervention operator, which forcibly sets variable X to x' by severing all incoming causal connections to X in the underlying model [1]. Three axioms govern atomic interventions: *effectiveness* ($\text{do}(X=x')$ successfully sets X to x'), *composition* (intervening on X while holding W at its natural value w produces the same outcome as $\text{do}(X=x')$ alone when $W_x = w$), and *reversibility* (algebraic reversal of intervention semantics within a single structural model). EXTp operationalizes effectiveness and composition; reversibility is inapplicable under linear-time execution and is declared as a concrete divergence point rather than silently elided (§V-D).

Halpern and Pearl’s actual-causation framework [2] specifies when a specific event is *the* cause of an observed outcome. The AC1 condition requires that both the candidate cause and its effect occur in the actual world; AC2 has two parts: AC2(a) requires counterfactual sensitivity — that under an alternative assignment to the cause, the effect changes — and AC2(b) requires a hold-fixed condition, that this sensitivity holds while keeping a specified set of other variables fixed at their actual values; AC3 requires minimality over the cause set. EXTp’s

¹The AppArmor confused-deputy cluster (9 CVEs); analyzed using EXTp’s vault infrastructure. Chronomaly (CVE-2025-38352, POSIX CPU timer UAF) is a scope boundary example, not a demonstration target; see §3 Out of Scope, item 1.

CST establishes AC1 and AC2 for every admissible atomic claim. AC3 (minimality) is outside CST v1.0 scope: CST produces *causal-link* claims, not *causal-explanation* claims in the Halpern-Pearl sense. The distinction is consequential but the omission is principled: CST’s downstream applications (mitigation verification and counterfactual fuzzing, §VII-B) require attribution at individual intervention points — “did $\text{do}(X=x')$ cause the divergence?” — not identification of minimal sufficient cause sets. A mitigation-verification workflow needs to know whether patching parameter X severs the causal pathway; whether $\{X\}$ is the minimal such set is operationally irrelevant. AC3 minimality becomes essential only when CST claims are used to construct human-readable explanations, a separate use case that EXTP does not target in v1.0.

Pearl’s faithfulness axiom requires a probability distribution to test conditional independence against; this presupposition holds in observational studies but not in EXTP’s trajectory-level deterministic setting. The implication for CST’s design is developed in §V-D.

III. THREAT MODEL

Attacker, Target, and Analyst

EXTP targets a specific analysis scenario: a security researcher has captured an exploit trace — a sequence of VM-exit events produced by guest-level code attempting privilege escalation or isolation bypass on the seL4 VMM — and wishes to determine which input parameters causally produced the observed outcome. The target is the seL4 VMM and its hosted guest workloads; EXTP does not model attacks against EXTP itself. The attacker is the guest executing the exploit; the analyst is EXTP.

EXTP makes no attempt to prevent the exploit from succeeding during the original capture. It operates *post-hoc*: given a trace T_{orig} of an exploit that has already executed, EXTP constructs counterfactual traces T_{cf} under controlled input modifications and issues causal attribution claims about the divergence.

Trusted Computing Base

EXTP’s trusted computing base (TCB) comprises the seL4 microkernel, the EXTP VMM implementation, and the underlying Intel VT-x hardware. Specifically:

- **seL4**: functional correctness and capability integrity as established by the L4.verified proof [5], [6].
- **Intel VT-x hardware**: VM-exit semantics and VMCS field correctness. Microcode revision 0x3e (Alder Lake, pre-Downfall); post-Downfall microcode alters $\hat{\sigma}_{\text{baseline}}$ characteristics and requires MI recalibration (§V-C); cross-microcode portability is out of scope (item 3 below).
- **EXTP VMM**: implementation in C, gcc 12.3 at -O2 (build hash in Appendix B); intervention machinery operates within $\mathcal{K}_{\text{extended}}$, with side-effect properties empirically bounded by CapSep (§V-A). EXTP’s source-level memory safety is not formally verified — a concrete TCB

risk; mitigation pathway via seL4’s Isabelle infrastructure is outlined in §VII.

Out of Scope

The following are explicitly outside EXTP’s threat model and formal coverage:

- **Multi-VCPU concurrent exploits**. Race-condition-dependent exploits (e.g., Chronomaly-class POSIX timer races) require non-deterministic scheduling interactions that break SOE’s replay determinism (A4). Multi-VCPU counterfactual replay is future work.
- **Silent microarchitectural channels**. Causal pathways that manifest exclusively in cache, branch predictor, or TLB state without producing 4-tuple or timing-above- Δ^* divergence fall outside A2’s observable surface. A future causal microarchitectural decomposition (CMD) property class would address this.
- **Cross-configuration portability**. CST’s formal assumptions are configuration-conditioned; claims do not transfer across hardware revisions, microcode updates, or VMM builds without recalibration. Recalibration requires re-establishing MI’s $\hat{\sigma}_{\text{baseline}}$ (§V-C) and CapSep’s Clopper-Pearson bounds (§V-A) on the target configuration; estimated effort: $N \geq 200$ cross-boot trials.

IV. EXTP FRAMEWORK

A. Architecture Overview

EXTP operates in three phases: *capture*, *replay*, and *analysis*. Figure 1 shows the architecture.

During **capture**, EXTP runs the exploit guest under the seL4 VMM. On each VM-exit, `extp_event_from_vmenter()` records the 4-tuple observable surface plus full general-purpose register state (rax-r15), per-event timing via bare RDTSC, and exit semantics (guest physical address, rflags, cr3, instruction length). The result is the original trace T_{orig} : a sequence of VM-exit events $[e_0, e_1, \dots, e_n]$, each carrying the observable surface and timing Δ_i .

During **replay**, EXTP re-executes the trace in a fresh boot session with a controlled modification. A VMCS snapshot (`extp_snapshot_save`, 23 fields) and EPT Copy-on-Write checkpoint (`extp_ept_cow_restore`) restore guest state to the pre-exploit configuration at each run. The intervention is applied just before VM-entry; the resulting trace T_{cf} is compared event-by-event against T_{orig} via `extp_replay_compare()` and `extp_determinism_score()`.

During **analysis**, the divergence detection function $D(T_{\text{orig}}, T_{\text{cf}}, i)$ identifies events where the 4-tuple or timing differs above Δ^* . If divergence is detected and the CST admissibility check passes (§V-D), EXTP issues a causal attribution claim $\langle \text{do}(X=x') \rightsquigarrow_i D \mid A; \mathcal{E} \rangle$ for that event, where $\mathcal{E}$ is the validity envelope constructed during the admissibility check.

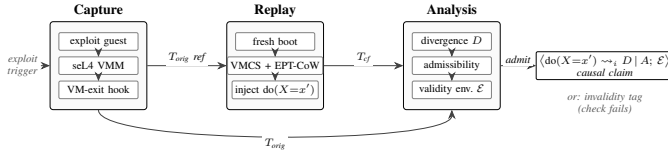


Fig. 1. EXTP architecture. The exploit guest produces T_{orig} during capture (VM-exit observable surface plus per-event timing). T_{orig} serves as the replay reference: in a fresh-boot session, the intervention $\text{do}(X=x')$ is injected at the designated event’s VM-entry boundary, producing T_{cf} . Divergence detection D fires on either 4-tuple difference (SOE clause) or timing excursion above Δ^* (MI clause). If divergence is detected and the four-step CST admissibility check passes, EXTP emits a causal attribution claim.

B. Trace Capture and Fresh-Boot Replay

EXTP’s replay fidelity rests on two restore mechanisms operating in concert. *VMCS-level restore* resets 23 VMCS fields covering guest control state, segment descriptors, and exception handling (full enumeration in Appendix C), plus RIP, RSP, RFLAGS, and CR3, placing the virtual CPU in a clean initial state. Non-restored fields are either constant across replay (host state) or read-only (derived state).

EPT Copy-on-Write restore reverts guest-modified physical pages to their checkpoint state. The mechanism operates via EPT permission downgrade: at checkpoint, all guest-writable pages are marked read-only in the EPT; the first guest write to each page triggers an EPT violation routed to `extp_ept_cow_handler()`, which allocates a shadow page, copies the original content, and remaps the guest’s view to the shadow. Restore reverts each EPT mapping to the original page and reclaims the shadow, returning the guest’s memory view to its pre-exploit state.

These two mechanisms handle the architectural state visible through the VT-x interface. Microarchitectural state — cache occupancy, branch predictor history, TLB entries, MSR-resident state — is not reset by VMCS reload. EXTP addresses this through *fresh-boot discipline*: each counterfactual replay T_{cf} executes in an independent boot session, not within the same boot as T_{orig} . Cross-boot microarchitectural state is not preserved across power cycles, providing a conservative reset guarantee. The cost is measurement overhead; within-boot replay with explicit microarch state reset machinery is future work.

Cross-boot replay’s adequacy as a microarchitectural reset mechanism is the empirical claim that SOE (§V-B) tests: $\text{score}_S = 1.0000$ over $N=34$ cross-boot trials in the absence of intervention is the operational evidence that fresh-boot discipline produces deterministic 4-tuple trajectories.

C. Intervention Machinery

EXTP’s intervention machinery modifies a designated parameter X to value x' at a specified event e_i during T_{cf} ’s execution. Table I summarizes the four validated intervention classes.

The classification into *state interventions* (input_value, reg_state; cache_state pending EPT warmup validation — see Table I) and *temporal interventions* (nopcount) reflects the

TABLE I
EXTP INTERVENTION CLASSES.

Class	Type	Status	Mechanism
nopcount	Temp.	Validated	Code patch, iter byte
register write	State	Implem.	vcpu->regs post-snap
input value	State	Implem.	Hypercall arg rewrite
cache preload	State	Future	EPT + warmup pass

primary expected observable surface: state interventions are designed to produce 4-tuple divergence; temporal interventions are designed to produce timing divergence above Δ^* . Both classes may trigger either clause of the divergence detector D ; the classification predicts, not constrains, the expected signal.

Intervention scope in CST v1.0 is *per-event*: X is set to x' exactly at event e_i and reverts thereafter; extended scopes are discussed in §VII.

Compound interventions — multiple modifications in the same T_{cf} — are modeled as an ordered atomic sequence. Commutativity is not assumed: $\text{do}(X_1=x'_1) \rightarrow \text{do}(X_2=x'_2)$ and $\text{do}(X_2=x'_2) \rightarrow \text{do}(X_1=x'_1)$ may produce different divergence patterns if X_1 ’s effect alters the state on which X_2 operates. Reordering analysis is a per-case investigation, not a framework-level guarantee.

EXTP’s admissibility check, detailed formally in §V-D, takes the following operational form in the framework: before issuing any causal claim, EXTP performs four steps: (1) the validity envelope $\mathcal{E}$ ’s tuple components match CST v1.0 coverage; (2) assumption validation references for A1–A4 are present and current; (3) divergence $D=\text{true}$ was actually observed; (4) X is within the validated intervention surface. Failure of any step produces a structured invalidity tag rather than a claim (§V-D).

V. FORMAL PROPERTIES

A. Capability Separation (CapSep)

Proof structure. CapSep is a hybrid structural-empirical claim: structurally proven on $\mathcal{K}_{\text{verified}}$ by inheritance from L4.verified, and empirically bounded on $\mathcal{K}_{\text{extended}}$ via Clopper-Pearson confidence intervals on observed zero-failure trials.

Claim. $\mathcal{K}_{\text{int}} \cap \mathcal{K}_{\text{fwk}} = \emptyset$, where $\mathcal{K}_{\text{int}}$ is the capability set held by EXTP’s intervention machinery and $\mathcal{K}_{\text{fwk}}$ is the capability set governing framework-internal state (VMM scheduler, EPT structures outside the intervention surface, capability registers not under intervention authority).

Two-layer structure. The disjointness claim has structurally different strength on two operation classes introduced in §II:

For $\mathcal{K}_{\text{verified}}$ operations, disjointness follows from seL4’s L4.verified proof: the Isabelle/HOL functional correctness theorem establishes external atomicity — that every capability operation in $\mathcal{K}_{\text{verified}}$ completes as an observable single step

with no intermediate state exposed. Because the intervention machinery’s authority set is a subset of $\mathcal{K}_{\text{extended}}$ and $\mathcal{K}_{\text{verified}} \cap \mathcal{K}_{\text{extended}} = \emptyset$, authority disjointness on $\mathcal{K}_{\text{verified}}$ is unconditional.

For $\mathcal{K}_{\text{extended}}$ operations (the VT-x extension API enumerated in Appendix A), disjointness is empirically bounded. Disjointness is tested by two complementary protocols, each supporting a distinct downstream claim.

Within-boot trials ($N=998$) measure authority leakage within a single execution session and support *comparator reliability*: the protocol confirms that the intervention machinery does not pollute its own observation surface when running in steady state. Observed failure rate is zero; Clopper-Pearson one-sided 95% upper bound on the failure rate is $\leq 0.37\%$.

Cross-boot trials ($N=34$) measure authority leakage across independent boot sessions and support *A4’s operational bound*: the protocol confirms that residual microarchitectural state from previous runs does not survive fresh-boot discipline. Observed failure rate is zero; Clopper-Pearson one-sided 95% upper bound is $\leq 10.4\%$. The wider cross-boot bound reflects the smaller sample (cross-boot trials are operationally expensive). The within-boot bound is the operational confidence value used in A1-modulo claims; the cross-boot bound underwrites A4’s replay-determinism antecedent (§V-D).

Pearl role. Pearl’s modularity axiom requires that $\text{do}(X=x')$ severs all incoming causal connections to X in the structural model. In EXTP’s setting, “incoming causal connections to X ” correspond to capability-mediated authority over X ’s state. Disjointness $\mathcal{K}_{\text{int}} \cap \mathcal{K}_{\text{fwk}} = \emptyset$ therefore establishes that intervention authority cannot reach X via any framework-internal pathway, satisfying modularity within the capability-mediated subset of causal connections. The *known gap* below clarifies that non-capability side-effect paths (cache, scheduler overhead) lie outside this subset.

Known gap. CapSep establishes *authority disjointness*, not *side-effect freedom*. The intervention machinery’s execution itself consumes microarchitectural resources — VMM scheduler time, cache lines, branch predictor history — that may produce indirect effects on framework state not governed by the capability system. These side-effect paths are outside the $\mathcal{K}_{\text{int}} \cap \mathcal{K}_{\text{fwk}}$ disjointness argument; their empirical boundedness is the subject of the Q4 future experiment (§VII). Until Q4 resolves, A1 (Pearl modularity; formally stated in §V-D) in CST is asserted modulo side-effects below Δ^* on framework observables, and this qualifier propagates: every CST claim’s validity envelope $\mathcal{E}$ carries an explicit A1-modulo flag; downstream tools consuming CST claims must check this flag before relying on causal attribution. All claims in this paper are issued under CST v1.0 and carry this flag unless Q4 resolves otherwise.

B. Strong Observational Equivalence (SOE)

Proof structure. SOE is a hybrid formal-empirical claim with two antecedents: a structural antecedent (Capability Separation, §V-A) establishing that replay and original execution operate under divergent authority, and an empirical antecedent

(comparator validity via four-field injection testing) establishing that the 4-tuple comparator is faithful to its specification.

Claim. Let E be a guest execution under EXTP’s VMM, and $R = \text{replay}(E)$ the corresponding replay execution. Then:

$$E \approx_S R \quad \text{iff} \quad \forall i : \text{observable}(E)_i = \text{observable}(R)_i \quad (2)$$

where the 4-tuple at event i is

$$\text{observable}(X)_i = \langle \text{rip}_i(X), \text{exit_reason}_i(X), \text{exit_qual}_i(X), \text{rax}_i(X) \rangle.$$

Empirical support. Across $N=998$ single-boot replay trials, $\text{score}_S = 1.0000$ (exact: zero failures on all four observable fields across all events in all runs). Across $N=34$ cross-boot trials, $\text{score}_S = 1.0000$ again. Clopper-Pearson one-sided 95% upper confidence bounds on the per-run failure rate are $\leq 0.37\%$ at $N=998$ and $\leq 10.4\%$ at $N=34$. The $N=34$ cross-boot bound is the conservative public-facing figure cited in the introduction; the tighter $N=998$ bound is the operational value. In either case, the “1.0000” framing is *high-confidence empirical*, not categorical.

Scope limitation. The observable surface covers RAX as the sole general-purpose register. Other GPRs (RBX, RCX, RDX, R8–R15) are captured internally but excluded from the formal surface for falsifiability: a surface defined over all 16 GPRs would require determinism claims over register state that EXTP’s current calibration does not support. Exploits whose divergence manifests exclusively in non-RAX registers are silent on the SOE surface — a scope boundary analogous to MI’s silent-leak class. Extension to the full register file is straightforward mechanically but requires N -trial recalibration; deferred (§VII).

Pearl role. SOE underwrites two assumptions: A4 (replay determinism) and, through its 4-tuple surface definition, A2 (observable surface coverage) — both formally stated in §V-D. SOE’s trajectory-level reproducibility is *strictly stronger* than Pearl’s probabilistic faithfulness axiom: faithfulness requires that conditional independences in an observed distribution correspond to d-separations in the underlying DAG, which presupposes a probability distribution. Under SOE’s deterministic replay, there is no distribution — trajectories are reproduced exactly, making probabilistic CI testing unnecessary. A4 therefore inherits a property stronger than faithfulness requires, which is why CST’s proof sketch (§V-D) does not appeal to CI testing at any step.

C. Measurement Interference (MI)

Proof structure. MI is an empirical claim grounded in two antecedents: (i) $\hat{\sigma}_{\text{baseline}}$ estimation from post-warmup zero-perturbation samples per workload class and instrument, and (ii) FPR validation against nominal α via Clopper-Pearson confidence intervals on observed false-positive counts.

Claim. For workload class L under timing instrument S , there exists a calibrated detection threshold $\Delta^* = z_{1-\alpha} \cdot \hat{\sigma}_{\text{baseline}}$ such that a per-event timing deviation above Δ^* is detectable with false-positive rate bounded by α (nominal 0.05). $\hat{\sigma}_{\text{baseline}}$ is the sample standard deviation of post-warmup

timing samples at zero perturbation (iter=0), computed within each experiment.

Timing instruments. EXTP supports two instruments. S_1 (LFENCE-sandwich-RDTSC) serializes instruction retirement before the TSC read, adding $\sim 3\times$ noise over S_2 (bare RDTSC) but eliminating out-of-order TSC read artifacts. S_2 is preferred for detection-threshold calibration; S_1 for cross-workload comparisons requiring serialization consistency.

Calibrated values. For the L_1 -pphase workload class under S_1 : $\hat{\sigma}_{\text{baseline}} = 5,518$ cycle ($n=108$, boot_log12, iter=0), slope $b = 4.481$ cycle/iter, $\Delta^* = 1.645 \times 5,518 = 9,077$ cycle. For L_2 -pphase under S_2 (boot_log15, $n=108$, iter=0): $\hat{\sigma}_{\text{baseline}} = 4,604$ cycle, $b = 15.194$ cycle/iter, $\Delta^* = 1.645 \times 4,604 = 7,574$ cycle. The $\hat{\sigma}$ confidence interval at $n=108$ is $[4,867, 6,371]$ cycle (95%), implying a Δ^* uncertainty range of approximately $+15\%/-12\%$ (asymmetric, reflecting the χ^2 CI structure on $\hat{\sigma}$; upper: $1.645 \times 6,371 = 10,480$ cycle; lower: $1.645 \times 4,867 = 8,006$ cycle). This is $\hat{\sigma}$ -noise dominated, not a functional-form mismatch (§VI).

Empirical FPR. Realized false-positive rate is workload-dependent. For L_1 -pphase under S_1 (boot_log8, $n=978$): 0.61% (Clopper-Pearson 95% CI $[0.22\%, 1.33\%]$). For L_1 -pphase under S_1 at finer grain (boot_log12, iter=0, $n=108$): 2.78% (CI $[0.58\%, 7.91\%]$). Both lie below nominal $\alpha=5\%$; the wider boot_log12 interval reflects the smaller n .

The use of $z = 1.645$ (Gaussian one-sided $\alpha=5\%$ quantile) for Δ^* calibration is a deliberate design choice, not a distributional assumption: when the empirical timing distribution has heavier-than-Gaussian tails (as observed under L_1 -pphase, where the empirical-to-nominal FPR ratio is $\approx 8.2\times$ for boot_log8), the Gaussian threshold over-estimates tail probability and therefore yields empirical FPR strictly below the nominal α . This is conservative in the false-positive direction by construction. The cost is reduced detection sensitivity for effects near Δ^* ; this trade-off is intentional and is consistent with EXTP’s broader emphasis on sound (rather than complete) attribution.

Scope: per-event detection. MI claims per-event detection-threshold validity. The detection criterion’s α is per-event: each event-indexed comparison carries nominal false-positive rate α . For a trace of N events, the expected false detection count under no perturbation is $N \cdot \alpha_{\text{realized}}$. Trace-level false discovery rate (FWER) is the downstream consumer’s aggregation responsibility; MI does not prescribe a correction method, as the appropriate correction depends on the consumer’s use case.

Pearl role. MI underwrites A3 (formally stated in §V-D): timing deviations above Δ^* are attributable to intervention rather than baseline noise, at the calibrated FPR. A3 is an EXTP-specific assumption addressing the empirical noise floor absent from Pearl’s idealized structural equations; it captures confounder boundedness in the timing domain and complements Pearl’s axioms rather than mapping directly to any one of them. MI’s Δ^* extends SOE’s 4-tuple surface (A2, §V-D) into the timing domain, enabling detection of interventions whose causal effect is temporal rather than structural.

D. Counterfactual Soundness Theorem (CST)

Proof structure. CST is the binding theorem of the property hierarchy: it formally proves that the three observable properties (CapSep, SOE, MI) compose into sound causal attribution. CST does not make new empirical claims; it asserts that CapSep + SOE + MI, in composition, are sufficient to ground Pearl-style causal attribution within enumerated epistemic bounds.

Theorem 1 (CST Composition Closure, atomic case). *Let T_{orig} and T_{cf} be counterfactual replays under a single configuration, with T_{cf} generated by atomic intervention $\text{do}(X=x')$. Let $A = (A_1, A_2, A_3, A_4)$ be the assumption envelope and $\mathcal{E}$ the validity envelope. Under A satisfied and $\mathcal{E}$ admitted, the atomic CST claim*

$$\langle \text{do}(X=x') \rightsquigarrow_i D \mid A; \mathcal{E} \rangle$$

is admissible iff $D(T_{\text{orig}}, T_{\text{cf}}, i) = \text{true}$ and the four-step admissibility check (§IV-C) passes. When admissible, the claim asserts that observable divergence D at event i is causally attributable to $\text{do}(X=x')$, without making claims about the microarchitectural mechanism by which the divergence arose. Furthermore, the admissible claim satisfies:

- (i) *Pearl’s effectiveness axiom: $\text{do}(X=x')$ sets X to x' ;*
- (ii) *Pearl’s composition axiom in a δ_{A1} -relaxed form: composition holds with $W_{\text{cf}} - W_{\text{orig}}$ bounded by δ_{A1} (CapSep §V-A) rather than exactly equal. The strict Pearl form is recovered in the $\delta_{A1} \rightarrow 0$ limit; CST v1.0 operates with $\delta_{A1} \leq 0.37\%$ (within-boot) / $\leq 10.4\%$ (cross-boot). The Q4 modularity experiment (§VII) targets strict-form recovery by validating side-effect freedom directly;*
- (iii) *Halpern-Pearl AC1: effect D occurs in the counterfactual trace T_{cf} ;²*
- (iv) *Halpern-Pearl AC2(a): effect D differs from T_{orig} under intervention, establishing counterfactual sensitivity. AC2(b) [witness fixity]: we construct the HP witness set $W = S_{\text{in}} \setminus \{\text{observable}(\cdot)_i\}$ from the validity envelope’s S_{in} specification and verify the two HP witness conditions — (W1) fixity enforceability discharged by A4, and (W2) the fixity-does-not-mask condition discharged jointly by Lemma 1 and Lemma 4. The witness-fixity construction is derived in full in Appendix D, Composition into Theorem 1, item (iv), which also refines the residual envelope per witness clause (structural vs. timing), with which is tighter depending on the calibration regime.*

Assumptions. Table II states the four assumptions formally. Each derives from an upstream property entry.

Proof argument structure: threat elimination. We adopt a threat-elimination structure (rather than direct antecedent→conclusion derivation) because the three threats correspond directly to the empirical gaps motivating EXTP’s

²AC1’s “actual world” in CST is T_{cf} — the world in which the intervention was applied and the divergence observed. This differs from the classical HP framing where the actual world is pre-intervention.

TABLE II
CST ASSUMPTIONS.

Assump.	Role	Source	Status
A1 Modul.	do() side-eff. free	CapSep	Mod. Q4
A2 Obs. comp.	Scope-compl.	SOE	Mod. silent
A3 Confound.	Noise $< \Delta^*$	MI	Calibrated
A4 Determ.	No coincidence	SOE	High-conf.

Source columns: CapSep (§V-A), SOE (§V-B), MI (§V-C). A2 silent-channel qualifier per §III. A4 confidence $N=34$.

three properties; a rigorous formal derivation following standard proof structure is in Appendix D. The proof addresses three threats in turn.

Threat 1 — Coincidence. Under A4 (replay determinism, grounded by SOE’s score_S = 1.0000 over $N=34$ cross-boot trials), spontaneous divergence on the 4-tuple surface is ruled out at false-positive rate zero: any 4-tuple divergence in T_{cf} relative to T_{orig} requires a trace-distinguishing input difference; under the single-configuration discipline, the only controlled trace-distinguishing input is $do(X=x')$. External factors (microcode jitter, thermal variance, hardware faults) are bounded by SOE’s empirical score_S = 1.0000 over $N=34$ cross-boot trials; attribution to $do(X=x')$ is the maximum-likelihood explanation within the calibrated confidence envelope. The timing clause admits a residual coincidence rate $\alpha_{realized}$, calibrated per workload class (0.61% under L_1 -pchase/ S_1 boot_log8; 2.78% under L_1 -pchase/ S_1 boot_log12; MI §V-C), incorporated into A3 as an explicitly bounded, not silenced, source of attribution uncertainty.

Threat 2 — Silent mechanism. Addressed by scope-honesty under A2. If a silent microarchitectural process triggered by $do(X=x')$ produces no 4-tuple delta and no timing deviation above Δ^* , then $D = \text{false}$ and CST issues no claim — scope-honest silence. If the silent process does produce observable divergence, $D = \text{true}$ and CST attributes correctly at the event-chain level, mechanism-agnostic: CST claims that $do(X=x')$ caused the observable divergence, not that it caused the underlying microarchitectural change. Silent processes that manifest neither clause of D are explicitly deferred to a future CMD property class (§VII).

Threat 3 — Confounder. Addressed by A1 (modularity, grounded by CapSep §V-A), modulo the known gap. Under A1, intervention execution does not modify framework-internal state outside X ’s capability authority scope; no third factor correlated with $do(X=x')$ can produce the divergence. Until the side-effect quantification experiment resolves (Q4; §VII), A1 is asserted modulo side-effects below Δ^* : framework-internal side-effects bounded below the detection threshold are absorbed into A3’s noise model rather than constituting unbound confounders (propagated per-claim via the A1-modulo flag, §V-A).

Compositional extension.

Proposition 1 (Sequential composition). $\langle do(X_1) \rightsquigarrow_{i_1} \dots \rightsquigarrow_{i_n} D \mid A; \mathcal{E} \rangle$ is admissible iff every atomic step is

TABLE III
PEARL/HP CORRESPONDENCE WITH EXPLICIT DEVIATION MODES.

Construct	EXTp realization	Status	Relationship
Modularity	CapSep: $\mathcal{K}_{int} \cap \mathcal{K}_{fwk} = \emptyset$	Holds mod A1	Relaxed- δ_{A1}
Effectiveness	Intervention (§IV-C)	Holds	Direct
Composition	Inherits A1 modularity	Holds mod A1	Relaxed- δ_{A1}
Reversibility	Linear-time exec.	Inapplicable	Inapplicable
do-calc. R1–3	Degenerate-determ.	Hold	Stronger
Faithfulness	Subsumed by A4 (SOE)	Subsumed	Stronger
HP AC1	D in T_{cf} at i	Direct	Direct
HP AC2(a)	D differs in T_{orig}	Direct	Direct
HP AC2(b)	Witness fix. via fresh-boot	Direct (App. D)	Direct
HP AC3 (min.)	Future work	Out of scope	Deferred

independently admissible under Theorem 1.

Proof sketch by induction on chain length: the base case is Theorem 1; the inductive step requires that A1’s framework-state isolation propagates across sequential interventions, which follows from CapSep’s per-operation disjointness — each $\mathcal{K}_{int}$ operation remains disjoint from $\mathcal{K}_{fwk}$ regardless of execution order. Commutativity is not assumed. Full proof in Appendix D.

Pearl/HP correspondence. Table III summarizes CST’s correspondence to Pearl’s structural causal model and Halpern-Pearl actual-causation conditions.

The *Relationship* column in Table III makes explicit the four distinct ways in which CST’s apparatus relates to Pearl’s classical framework. *Direct* entries hold as straightforward correspondences without modification (effectiveness; HP AC1, AC2(a), AC2(b)). *Stronger* entries are properties that EXTp’s deterministic setting satisfies in a form strictly stronger than Pearl’s probabilistic original — faithfulness is subsumed by SOE’s trajectory-level determinism (§V-B), and do-calculus rules R1–R3 reduce to trajectory equality testing under A4. *Relaxed- δ_{A1}* entries are properties that hold in an approximate form bounded by CapSep’s empirical envelope (§V-A): modularity and composition are established to within δ_{A1} rather than exactly, with the strict Pearl form recovered in the $\delta_{A1} \rightarrow 0$ limit (which Q4 future work targets, §VII). *Inapplicable* entries are Pearl constructs whose preconditions do not arise in EXTp’s setting — reversibility requires algebraic intervention reversal within a single structural model, which has no analog under linear-time bare-metal execution. We declare these four modes explicitly rather than collapsing them under a single “holds modulo” qualifier; downstream consumers of CST claims can therefore distinguish strengthening from relaxation when assessing claim transportability.

AC3 (minimality over the cause set) is outside CST v1.0 scope: CST issues *causal-link* claims — attributing observable divergence to a named intervention — not *causal-explanation* claims identifying the minimal sufficient cause set. Extension

to AC3 requires a search procedure over candidate intervention sets and is deferred.

VI. EVALUATION

A. Experimental Setup

All experiments run on bare-metal Intel 12th Gen Core (Alder Lake, microcode 0x3e, ASUS Z690 motherboard, UEFI firmware) with no host OS. The seL4 VMM runs directly on hardware; the guest executes under EXTp’s VT-x control loop. Serial console infrastructure (DIGITUS USB-Serial FT232RL, Digitus PCIe Serial DS-30000-1, RS-232 null modem cable) provides out-of-band I/O during bare-metal runs. No network connectivity is present during any experiment.

All timing measurements use bare RDTSC (S_2) for detection-threshold calibration and LFENCE-sandwich-RDTSC (S_1) for cross-workload comparisons. EXTp’s VMM is compiled with gcc 12.3 at -O2 (build hash in Appendix B).

B. Privilege-Check-Bypass Demonstration

The demonstration exercises EXTp’s full attribution pipeline on a synthetic capability-bypass exploit pattern modeled on the CrackArmor confused-deputy class (§I, C4). *A synthetic analog is used rather than a real CrackArmor instance for a soundness-preserving reason, not a convenience one:* real CrackArmor exploits exercise multi-VCPU code paths (concurrent privilege-check races between cooperating threads) that lie outside CST v1.0’s single-VCPU envelope (§III, Out of Scope item 1). Running EXTp against a real CrackArmor trace would produce admissibility-check failures at step (1) — envelope mismatch — and would not yield a CST claim under any intervention, by design. The synthetic analog isolates the structural pattern CrackArmor exploits (capability-token forgery producing privileged-path divergence) within the single-VCPU regime where A4’s replay determinism holds, allowing the full attribution pipeline to execute end-to-end.

The scenario preserves the confused-deputy causal structure of the original CVE class — a token-mediated privilege check that, when bypassed, branches execution into a privileged path — while maintaining the single-VCPU determinism CST v1.0 requires. It produces both a clean SOE-clause divergence signal at event 1 and an empirical surface on which the timing clause and the A1-modulo bound can be independently observed (§VI-B, subsequent subsections).

1) *Scenario:* A 29-byte synthetic guest binary at GPA 0x10000 executes a control flow that branches on a capability token held in register `r8`.³ The default path (`r8 = 0`) executes a benign CPUID at offset 13 (RIP 0x1000d, RAX cleared); the privileged path (`r8 = 0xDEAD`) executes CPUID at offset 22 (RIP 0x10016, RAX 0x80000008). Table IV shows the predicted and observed divergence at event 1.

³`r8` is used rather than `rax` because the VMM’s CPUID handler deterministically writes the “EXTp” vendor ID into `rax-rdx` on every CPUID exit (§IV), clobbering those registers; `r8` is preserved across CPUID handling, making it a faithful carrier of the capability-token state across event boundaries.

TABLE IV
DEMO DIVERGENCE PREDICTION (MATCHES OBSERVATION ACROSS ALL $N=200$ POST-WARMUP TRIALS, BARE-METAL INTEL ALDER LAKE PER §VI).

Event	$T_{\text{orig}} (r8=0)$	$T_{\text{cf}} (r8=0xDEAD)$
0	RIP 0x10000, CPUID, RAX 0	identical
1	RIP 0x1000d, CPUID, RAX 0	RIP 0x10016, CPUID, RAX 0x80000008
2	RIP 0x10018, VMCALL	identical

The intervention `do(r8=0xDEAD)` fires immediately after the CPUID handler completes for event 0, via a second `WriteRegisters` call that overrides `r8` while preserving all other registers from the handler’s state. The guest then executes `cmp $0xDEAD, %r8` at offset 2 with the forged token, takes the conditional branch, and follows the privileged path through event 1’s CPUID. Event 2’s VMCALL terminates both traces identically (post-divergence reconvergence).

2) *SOE-clause divergence:* Across $N=100$ same-condition pairs and $N=100$ cross-condition pairs (single-boot run, 222 total runs minus 22 framework- and inner-warmup-discarded samples), the four-step admissibility check (§IV-C) passes for every trial:

- (1) Envelope match: instrument S_1 , workload class *synthetic-guest*, intervention scope per-event at event 0, hardware identity Alder Lake / microcode 0x3e / Z690.
- (2) Assumption validation: A1–A4 references current; A1 carries the A1-modulo flag (§V-A).
- (3) Surface check: $D(T_{\text{orig}}, T_{\text{cf}}, 1) = \text{true}$ via the SOE clause — RIP and RAX both diverge.
- (4) Intervention surface: `r8` $\in \{\text{reg_state}\}$ intervention class.

The admissible CST claim emitted in every trial is:

$$\langle \text{do}(r8=0xDEAD) \rightsquigarrow_1 D \mid A_1^{\text{mod}}, A_2, A_3, A_4; \mathcal{E}_{v1.0} \rangle$$

The SOE clause fires with false-positive rate zero (deterministic 4-tuple equality under A4’s score_S=1.0000 guarantee): the RIP delta 0x1000d $\rightarrow$ 0x10016 and RAX delta 0x0 $\rightarrow$ 0x80000008 are observed in every cross-condition pair, and zero divergences are observed in any same-condition pair.

3) *Timing-clause behavior and the A1-modulo bound:* The same demonstration was run under two implementations of the intervention machinery, exposing the timing clause’s sensitivity to the A1-modulo gap (§V-A).

a) *Silent hook (production):* The deployed implementation gates all informational output from the intervention apparatus (error paths excepted). Across $N=100$ per condition, the synthetic workload’s baseline timing standard deviation is $\hat{\sigma}_{\text{baseline}} = 6,237$ cycle (Clopper-Pearson via χ^2 CI: [5,476, 7,245], half-width $-12.2\% / +16.2\%$). The realized

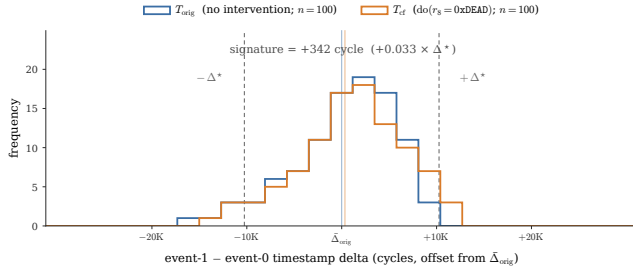


Fig. 2. Silent-hook demonstration: timing distribution of T_{orig} vs T_{cf} ($N=100$ per condition). The intervention signature (+342 cycle, $0.03 \cdot \Delta^*$) is dwarfed by the detection threshold (dashed red, $\pm\Delta^* = \pm 10,259$ cycle). Distributions overlap heavily, consistent with A1-modulo holding under the silent implementation.

threshold is $\Delta^* = 1.645 \cdot 6,237 = 10,259$ cycle. The intervention signature is

$$\overline{T}_{\text{cf}} - \overline{T}_{\text{orig}} = +342 \text{ cycle} \approx 0.03 \cdot \Delta^*$$

(≈ 70 ns at 4.9 GHz; one extra `WriteRegisters` syscall accounts for this magnitude). The timing clause of D is **sub-threshold** in every trial: the SOE clause is the sole divergence signal, and the A1-modulo bound holds at paper-grade N . Figure 2 shows the timing distribution.

b) Naïve hook (counter-demonstration).: An earlier implementation of the same intervention included an informational `printf` in the hook’s success path. Under this implementation, at $n=23$ per condition, the intervention signature is

$$\overline{T}_{\text{cf}} - \overline{T}_{\text{orig}} = +26,617,599 \text{ cycle} \approx 2,595 \cdot \Delta^*$$

where $\Delta^* = 10,259$ cycle is the authoritative threshold calibrated under the silent-hook implementation (§VI-B above; the naïve-hook trace’s own sample standard deviation 7,727 cycle reflects an A1-modulo-broken state contaminated by `printf` I/O latency and is not used as a detection threshold). The magnitude 26.6M cycle ≈ 5.43 ms at 4.9 GHz is itself diagnostically informative: at 115200-baud synchronous serial output, ~ 60 characters of `printf` yield ~ 5 ms of I/O latency, matching the observed signature. The SOE clause fires identically in both implementations (the 4-tuple delta is determined by the guest’s branch target, not by hook latency), but the timing clause fires for the *wrong reason* under the naïve hook: not because the guest’s causal effect is detectable on the timing surface, but because the intervention apparatus’s own execution cost exceeds Δ^* by three orders of magnitude. Figure 3 shows the contrast.

4) Reading: the framework detects its own A1 violations: This bifurcation is, on its face, a story about instrumentation discipline: silent hooks pass the A1-modulo check; verbose hooks do not. But the structural point for CST is sharper. The naïve-hook data is not a bug-fix anecdote; it is an *operational test* of CST’s own assumption envelope. The naïve hook produces an intervention apparatus whose execution cost on the timing surface (+26.6M cycle) demonstrably exceeds Δ^* — precisely the condition under which A1’s modulo qualifier

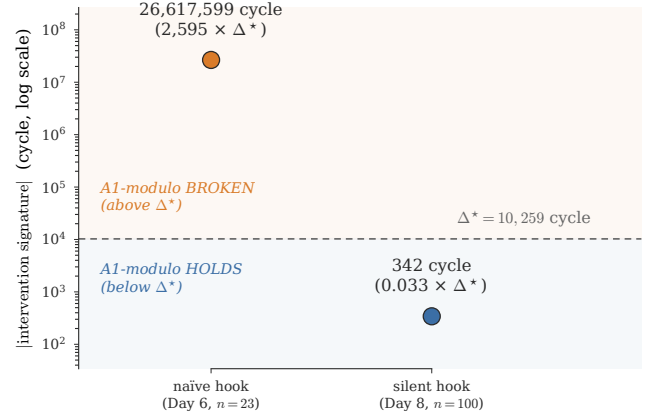


Fig. 3. Intervention signature magnitude under two hook implementations, compared against the authoritative detection threshold $\Delta^* = 10,259$ cycle (silent-hook calibration, dashed). The naïve hook ($n=23$, Day 6): signature +26,617,599 cycle $\approx 2,595 \cdot \Delta^*$, A1-modulo *broken* (printf I/O latency dominates the timing surface). The silent hook ($n=100$, Day 8): signature +342 cycle $\approx 0.033 \cdot \Delta^*$, A1-modulo *holds* (intervention’s own execution cost is well below the detection threshold). The SOE-clause divergence is identical in both implementations; only the timing clause distinguishes them.

fails. The framework’s timing clause correctly fires; CST’s admissibility check correctly attaches the A1-modulo flag; downstream consumers reading that flag are correctly warned that the timing surface contains a machinery contribution. The silent hook restores A1-modulo and returns the timing clause to its design-intended role (intervention-effect detection rather than machinery-cost detection). The SOE clause is invariant across both implementations because 4-tuple determinism is a property of the guest’s causal structure, not the hook’s instrumentation overhead — the structural feature that makes SOE the robust attribution signal under A4.

5) Counterfactual sweep: causal coverage map: The single-intervention demonstration above establishes that `do( $X=r_8=0x\text{DEAD}$ )` produces an admissible CST claim. To test whether the same admissibility machinery correctly handles the broader fuzzing-style sweep CST is designed to support (§VII-B), we ran a 33-value sweep over $r_8 \in S_{r_8}$ on the same bare-metal configuration, where S_{r_8} contains the validated exploit value (`0xDEAD`), 16 low-byte values (`0x0–0xF`) spanning the structural-collision region, 14 wide-pattern values (`0x1111, 0x2222, ..., 0xEEEE`), and the two semantic distractors `0xCAFE` and `0xBEEF`. Each of the 33 r_8 values is applied as an *independent atomic intervention* at event 0; the sweep is a *parallel ensemble* of Theorem 1 instances rather than a sequential composition under Proposition 1. Each run begins from an independent fresh-boot state, applies a single `do( $r_8=v$ )` at the designated event, and is evaluated against T_{orig} by the same atomic admissibility check (§IV-C). The δ_{A1} modulo bound therefore applies per-instance and does not accumulate across the sweep. Each value was applied twice (66 T_{cf} runs total) under the silent-hook implementation.

Result. The divergence count is 1/33, matching the binary scenario design exactly: only $r_8 = 0x\text{DEAD}$ triggered the privileged path (RIP `0x10016`, RAX `0x80000008`, SOE

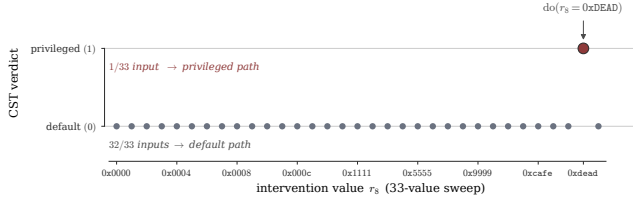


Fig. 4. Counterfactual fuzzing causal map. CST verdict for each of the 33 swept r_8 values: 32 default-path inputs (gray, $D=false$, SOE clause silent, no admissible divergence claim) versus the single privileged-path input $0xDEAD$ (dark red, $D=true$, SOE clause fires, admissible CST claim emitted). The envelope over this exact 32/1 sweep is machine-checked (dead_sweep_envelope_vs_naive_sweep, Table VI). The 33-input sweep demonstrates the falsifiability primitive: the framework’s verdict separates structurally on D ’s observable predicate, rather than on heuristic crash signatures or aggregate metrics. This is mechanism evidence, not a fuzzing benchmark; sweep coverage and discovery-rate evaluation are out of scope for CST v1.0.

clause firing); all 32 other values followed the default path (RIP $0x1000d$, RAX 0, SOE clause silent). Within-input reproducibility was 100%: every r_8 value produced an identical event-1 4-tuple across both of its repeat passes, extending A4’s determinism guarantee from the no-intervention baseline ($score_S=1.0000$ over $N=34$ cross-boot, §V-B) to the intervention-applied case across the full sweep. Figure 4 visualizes the verdict map.

This map instantiates the witness-refined ensemble envelope of Appendix D: one admissible claim (structural clause) among 32 scope-honestly silent inputs that emit no claim, so the sweep’s aggregate envelope is that of a single structural claim (1.35% within-boot), not the $33\times$ figure a per-input accounting would suggest. The instantiation is machine-checked (§VII-C).

Reading. The sweep is the operational instance of what CST-grounded fuzzing produces: a per-input causal coverage map where each point is labeled by admissibility verdict rather than crash signature. Two structurally distinct failure modes (silent admissibility with $D=false$ versus admissible divergence with $D=true$) are reported as distinct output classes by the same framework, with no heuristic acceptance threshold introduced between them. The single-shot $n=2$ -per-value confidence is mechanically modest; the structural claim is that admissibility is a falsifiable classifier, separately verifiable per input. The downstream implications for counterfactual fuzzing are discussed in §VII-B.

C. Measurement Characterization

Table V reports $\hat{\sigma}_{\text{baseline}}$, Δ^* , realized FPR, and $score_S$ across the three workload classes characterized to date: the two pchase reference workloads from MI calibration (§V-C) and the synthetic-guest workload introduced by the demonstration (§VI-B).

The synthetic-guest $\hat{\sigma}_{\text{baseline}}$ (6,237 cycle) sits within $\approx 13\%$ of the L_1 -pchase baseline (5,518 cycle), consistent with both workloads inhabiting the same noise-floor regime characteristic of bare-RDTSC sampling under microcode $0x3e$; the modest elevation is attributable to the synthetic binary’s conditional

TABLE V
MEASUREMENT CHARACTERIZATION.

Workload	Instr	$\hat{\sigma}$	Δ^*	FPR	$score_S$
L_1 -pchase	S_1	5,518	9,077	0.61%	1.0000
L_2 -pchase	S_2	4,604	7,574	2.78%	1.0000
Synth. guest	S_1	6,237	10,259	subthresh.	1.0000

Pchase rows: $N=998$ within-boot. Synthetic-guest row: $n=100$ per condition, χ^2 95% CI on $\hat{\sigma}$: [5,476, 7,245], half-width $-12.2\%/+16.2\%$. “Subthresh.” for synth. FPR: silent-hook intervention signature +342 cycle ($0.03 \cdot \Delta^*$) remains below detection threshold.

branch (cmp + je) introducing additional μop scheduling variance over pchase’s straight-line loop. The Day 6b cross-boot probe ($n=69$, three independent boots in alternation) yielded $\hat{\sigma}_{\text{baseline}} = 5,614$ cycle for the same synthetic workload, agreeing with the Day 8 single-boot $\hat{\sigma}_{\text{baseline}} = 6,237$ within χ^2 CI overlap — providing $N=3$ cross-boot replication for the synthetic workload class alongside the $N=34$ cross-boot result for pchase.

$score_S = 1.0000$ across $N=34$ cross-boot trials (Clopper-Pearson 95% upper bound $\leq 10.4\%$) confirms that fresh-boot discipline produces deterministic 4-tuple trajectories for the characterized workload classes.

D. Limitations

Four concrete limitations bound the current evaluation.

$N=34$ cross-boot sample. The public-facing $score_S$ bound ($\leq 10.4\%$) is wide, reflecting the 34-trial sample. The operational within-boot bound ($\leq 0.37\%$, $N=998$) is tighter but does not cover cross-boot microarchitectural variation. Expanding N is straightforward mechanically; the primary cost is machine time.

$\hat{\sigma}$ confidence at $n=108$. The Δ^* uncertainty range of $+15\%/-12\%$ (asymmetric χ^2 CI on $\hat{\sigma}_{L_1\text{-pchase}}$) means detection sensitivity for effects near Δ^* carries non-trivial uncertainty. Per-experiment recalibration at larger n is deferred.

Workload-class calibration scope. All $\hat{\sigma}_{\text{baseline}}$ and FPR calibration in this paper is established for the L_1 -pchase and L_2 -pchase guest workload classes (plus the synthetic-guest class used in §VI-B). LLC-resident and DRAM-resident workload classes constitute a separate experimental track — Δ^* calibration for these classes is mechanically straightforward (re-running the MI calibration protocol of §V-C with the new workload binary) but has not been performed for this submission. CST claims issued in this paper are scoped to the calibrated workload classes via the validity envelope $\mathcal{E}$ ’s `wl-class` field (§IV); a claim issued under `wl-class = L1-pchase` carries no implicit transportability guarantee to `wl-class = LLC-resident`. Adding LLC- and DRAM-resident calibrations to extend the validated envelope is future work; the soundness machinery is invariant under such extensions, only the calibration constants change.

Q4 modularity gap. A1 (modularity) is asserted modulo side-effects below Δ^* until the Q4 experiment validates strict side-effect freedom. Every CST claim issued under this evaluation therefore carries the `A1-modulo` flag.

VII. DISCUSSION

A. Scope Boundaries and Extensions

Multi-VCPU exploits. Chronomaly-class race-condition exploits (CVE-2025-38352, POSIX CPU timer UAF [7]; declared scope boundary, §III) and KVM SVM INT3 reinjection bugs (CVE-2025-68259 [8]) depend on non-deterministic scheduling interactions between concurrent vCPUs: in the former, a TOCTOU race between `handle_posix_cpu_timers()` and `posix_cpu_timer_del()`; in the latter, a vCPU executes an INT3 byte while another CPU is concurrently text-poking the remainder of the instruction. EXTp’s A4 (replay determinism) presupposes single-VCPU VM-exit trace capture; concurrent vCPU interleavings produce non-reproducible traces under fresh-boot replay. Extending CST to multi-VCPU requires augmenting A4 with an explicit interleaving model — either a deterministic-scheduler constraint (every replay reproduces the original schedule) or a probabilistic equivalence relation over the schedule space — analogous to CertiKOS’s concurrent layer verification [9]. This is a designated future direction; v1.0 declines to issue CST claims over multi-VCPU traces rather than attempt unsound attribution.

Silent microarchitectural channels (CMD). EXTp’s A2 observable surface (4-tuple plus timing above Δ^*) does not cover causal pathways that manifest exclusively in cache state, branch predictor history, or TLB entries without producing a 4-tuple delta or timing excursion. Training Solo and VMScope (§VIII) are concrete instances. A planned Causal Microarchitectural Decomposition (CMD) property class would extend A2 to PMU-counter-based surfaces (LLC miss rate, branch misprediction rate, iTLB miss rate), with per-channel calibrated detection thresholds analogous to MI’s Δ^* . CMD validation requires (i) per-channel $\hat{\sigma}_{\text{baseline}}$ estimation under the same fresh-boot discipline used for timing (§V-C), and (ii) cross-channel independence testing to determine whether multiple PMU channels admit composition without inflating false-positive rate. The protocol is the SOE methodology applied per-channel.

A1-modulo as a methodological invariant. The naïve/silent bifurcation in §VI-B illustrates that A1-modulo is not a one-time calibration but a discipline that applies to every extension of the intervention surface. Each new intervention class introduced in future work — per-window scope, global scope, CMD-grounded interventions, multi-VCPU — must re-establish that the intervention apparatus’s framework-observable cost remains below the relevant detection threshold. The Q4 modularity experiment (below) is the prototype protocol; subsequent extensions inherit its structure.

Intervention scope extensions. CST v1.0 covers per-event intervention scope only. Per-window scope (sustaining a modification across a contiguous event range) and global scope (sustaining across the entire trace) require A1’s modularity bound to hold under *sustained* rather than instantaneous in-

tervention. The Q4 modularity experiment is a prerequisite: per-window scope admissibility cannot be asserted without first validating that extended intervention duration does not amplify framework-observable side-effects above Δ^* .

Q4 modularity validation. A1’s current modulo-status (the `A1-modulo` flag carried by every CST claim in this paper) will be resolved by a dedicated experiment: measuring framework-observable state deltas (VMM scheduler dispatch timing, EPT modification counts, capability register state) under each validated intervention class, calibrated against per-observable noise floors. If side-effects are validated below Δ^* across all framework observables, A1 is promoted to strict form in CST v1.1 and the `A1-modulo` flag is retired. If not, the affected observable classes are added to $\mathcal{S}_{\text{out}}$, narrowing the validity envelope rather than weakening the soundness claim.

AC3 minimality. Halpern-Pearl AC3 (§II) requires the cause set to be minimal: no proper subset of the cause variables is sufficient. CST v1.0 issues *causal-link* claims and explicitly defers *causal-explanation* claims. Extension to AC3 requires a search procedure over candidate intervention subsets, which is mechanically straightforward but computationally non-trivial; for k candidate variables a naïve search is $O(2^k)$ replays. Group-testing strategies analogous to AID’s approach [3] are the natural starting point.

B. Downstream Applications of CST

EXTp’s CST-grounded causal attribution enables two downstream application classes that conventional differential analysis cannot support without falling back on heuristic acceptance criteria.

Counterfactual fuzzing. Conventional fuzzing treats divergence as a binary signal: a crash, a hang, or a sanitizer trip distinguishes interesting inputs from uninteresting ones. CST-grounded fuzzing extends this with an admissibility filter: each candidate intervention either produces a CST-admissible causal-link claim or it does not, and the framework reports the distinction. The output is a *causal coverage map* — intervention points indexed by validity envelope rather than by crash signature. §VI-B5 demonstrates the elementary instance: a 33-value sweep over r_8 correctly separates the single privileged-path input from 32 default-path inputs by the same admissibility machinery, with no heuristic acceptance threshold between the verdict classes. This is a demonstration of the falsifiability primitive on which a fuzzer-scale extension could build, not a fuzzing benchmark. Two failure modes that conventional fuzzing conflates are separated by the admissibility filter: behavioral coverage without causal attribution (divergence observed but admissibility fails, e.g., under A1-modulo violation, as in the naïve-hook arc of §VI-B) is distinguished from causal coverage without divergence (admissibility holds but $D = \text{false}$, indicating an intervention with no observable effect on this surface, as in the 32 default-path inputs of §VI-B5). The first signals a need for instrumentation discipline; the second signals a need for surface extension (potentially via CMD).

Mitigation verification. Once a patch is applied to the guest binary, counterfactual replay of the original exploit trace under the patched binary tests whether the causal pathway has been severed. The resulting CST claim has the same structural form but with $D = \text{false}$: $\langle \text{do}(X=x'_{\text{exploit}}) \rightsquigarrow_i \neg D \mid A; \mathcal{E} \rangle$. The patch is verified to sever the causal pathway iff (i) admissibility holds (envelope match, assumption validation, intervention surface) and (ii) no divergence is observed. Unlike behavioral regression testing, which can fail silently when the patched binary exercises a different code path incidentally, this verification is causal: the patch is attributable as the cause of non-divergence.

C. Isabelle Formalization Pathway

The CST proof of §V-D and Appendix D is stated in standard mathematical notation; its *conditional core* has additionally been mechanized in Isabelle/HOL. We distinguish what that establishes from what it does not.

What is machine-checked. The development formalizes the observable surface, the event trace, the divergence predicate D , the validity envelope, the intervention model with $\text{do}(X=x')$ given state-update semantics, and the four-step admissibility check (§IV-C). With A_1 – A_4 as hypotheses it then proves the $\Rightarrow$ direction of Theorem 1 in full: Pearl effectiveness, the propagation of the three bounding lemmas through the union bounds (their content enters as the operational meaning of A_1 , A_3 , A_4 in the locale, not as derived facts), the δ_{A_1} -relaxed composition axiom in its $W/\mathcal{K}_{\text{fwk}}$ form (holding W at its T_{orig} values leaves the outcome of $\text{do}(X=x')$ unchanged outside W and perturbed by at most Δ^* per variable, with the strict form recovered at $\delta_{A_1}=0$), the Halpern-Pearl AC2(b) witness construction with its union bound, the per-witness envelope refinement and its regime criterion, Lemma 3 in its completeness direction (effect implies divergence and emission; the converse is definitional), and Proposition 1’s sequential composition with explicit trace threading (each step’s baseline is the previous step’s T_{cf}) together with its parallel-ensemble form. The development (available at <https://github.com/GITHUB-USER/extp-artifact>) closes under Isabelle’s kernel with no unproved obligations, and its assumptions are shown consistent by a model, so the theorems are not vacuous; the $\Leftarrow$ direction is, as in Appendix D, a property of the emission code true by construction.

What is not. Mechanizing the conditional core does not discharge the antecedents, and two layers remain open, neither reducible to a missing tactic.

First, an *abstraction-layer gap*. seL4’s Isabelle machine model [6] is defined over abstract event traces at the kernel-API level; EXTp’s surface is defined over VT-x micro-architectural state (VMCS fields, EPT mappings, register file). Bridging them requires a lifting from the seL4 abstract machine state to EXTp’s concrete VM-exit surface — and the equivalence $\approx_S$ defined at that granularity — which does not exist in the L4.verified harness and would be a standalone contribution. EXTp’s intervention machinery moreover sits in the VMM layer *above* seL4’s verified scope, so this leverages

the capability-model formalization rather than extending verified theorems; it is a multi-year effort, and even completed it would discharge A_1 only for $\mathcal{K}_{\text{verified}}$, since $\mathcal{K}_{\text{extended}}$ lies outside L4.verified by construction (§II) and stays empirical.

Second, an *empirical-to-structural lifting* question. A_4 ’s content is a probability bound (score_S=1.0000 observed; Clopper-Pearson per-run failure rate $\leq 10.4\%$ at $N=34$); representing this as a structural predicate in set-theoretic HOL is an open design choice (a probabilistic HOL extension, an axiomatic envelope layer, or a hybrid). Our mechanization sidesteps it by carrying the bounds as abstract reals: Isabelle establishes the conditional soundness theorem, and calibration data supplies the antecedents. The open question is the *interface format* between calibration and Isabelle’s term language, not whether conditional soundness is expressible.

VIII. RELATED WORK

A. Counterfactual and Differential Analysis

The closest prior work to EXTp is AID [3] (Adaptive Interventional Debugging, SIGMOD 2020), which applies Pearl’s do-calculus to debug intermittent failures in database applications via runtime interventions. AID combines causal analysis, fault injection, and group testing to identify root causes of nondeterministic failures. EXTp and AID share the Pearl-grounded intervention philosophy, but differ on three dimensions: AID targets database application failures in a probabilistic, multi-run statistical setting; EXTp targets hypervisor exploit execution on a formally verified microkernel in a deterministic single-configuration replay setting; and AID does not provide a formal soundness theorem for its causal attributions. EXTp’s CST is the first soundness theorem for causal exploit attribution that combines two ingredients absent from AID: a formally verified computational substrate (seL4’s L4.verified [6]), and admissibility conditioned on an enumerated antecedent envelope (A_1, A_2, A_3, A_4) with bounds propagated per-claim through the validity envelope $\mathcal{E}$. The distinction is not mechanization alone — CST’s conditional core is machine-checked, though its empirical antecedents remain calibration-supplied (§VII-C) — but the conditioning structure: AID issues bounded probabilistic attributions; CST issues attributions whose bounds are conditioned on a named, falsifiable set of structural and empirical antecedents.

CacheWarp [10] (USENIX Security 2024) demonstrates software-based fault injection on AMD SEV-ES/SNP by reverting modified cache lines to stale state, producing controlled execution divergence. CacheWarp uses controlled state perturbation to demonstrate AMD TEE integrity violations, sharing the surface-level pattern of intervention-induced divergence with EXTp but operating as an attack methodology rather than an analysis framework, and without a causal formalism. WeSee [11] (IEEE S&P 2024) and Heckler [12] (USENIX Security 2024) inject malicious interrupts into AMD SEV-SNP and Intel TDX VMs to induce arbitrary register and memory modifications. WeSee and Heckler exploit the interrupt injection interface as an attack vector, with the hypervisor as adversary and the VM as target. EXTp inverts this stance:

the VMM is the analyst, the intervention is methodologically controlled, and the goal is causal attribution rather than security violation. The interrupt injection surface motivates future extension of EXTP’s intervention surface to cover #VC and external interrupt boundaries (§VII).

Soloviev and Halpern [4] (arXiv 2021) extend the Halpern-Pearl framework to capture nested causal statements about security properties; Soloviev, Balliu, and Guanciale [13] (IEEE CSF 2024) introduce a peer-reviewed modal logic framework for security properties capturing confidentiality, integrity, and robust declassification variants. Both are conceptual or specification-level frameworks rather than operational systems. To our knowledge, no prior peer-reviewed work applies Pearl’s causal apparatus to exploit replay with a formal soundness proof over a verified substrate — the gap that EXTP closes.

B. Hypervisor Exploit Surface

CVE-2024-21305 [14] is an HVCI security feature bypass in Hyper-V enabling arbitrary kernel-mode code execution via non-secure VMCS configuration. The divergent execution path structure — EPT permission misconfig producing observable VM-exit differences — is the structural analog of EXTP’s privilege-check-bypass demonstration (§VI-B). CVE-2025-22224 (and -22225/-22226) [15] is a VMware ESXi guest-to-host escape chain (VMSA-2025-0004) demonstrating that VMM implementation bugs constitute an out-of-scope causal pathway for EXTP’s current framework.

Training Solo [16] (VUSec, IEEE S&P 2025) presents self-training Spectre-v2 attacks on Intel CPUs bypassing existing domain isolation mitigations, uncovering two hardware issues (CVE-2024-28956, CVE-2025-24495) that break guest-to-hypervisor isolation. VMScope [17] (ETH Zurich, CVE-2025-40300, Sep 2025) exploits incomplete branch predictor isolation across guest/host boundaries on AMD Zen CPUs. Both demonstrate branch predictor state as a causal pathway falling within EXTP’s declared silent microarchitectural channels scope boundary (§III). The CMD property class (§VII) is designed to bring this class — alongside cache, TLB, and other microarchitectural side-state — into a future observable surface; Training Solo and VMScope are concrete instances motivating CMD’s design priorities.

XSA-472 [18] (Xen Viridian interface, Sep 2025) reports a cluster of three vulnerabilities. The two NULL pointer dereferences (CVE-2025-27466, CVE-2025-58142) represent the class of framework-internal isolation violations that CapSep’s disjointness requirement ($\mathcal{K}_{\text{int}} \cap \mathcal{K}_{\text{fwk}} = \emptyset$) is designed to prevent. The race condition (CVE-2025-58143) falls within EXTP’s multi-VCPU scope boundary (§III); A4 deterministic replay requires future extension before such cases become tractable.

C. Formal Methods for Hypervisors

seL4 [5], [6] provides EXTP’s formal trust anchor. CertiKOS [9] (Gu et al., OSDI 2016) verifies a concurrent OS kernel with fine-grained locking in Coq, including the mCertiKOS hypervisor extension. EXTP’s choice of seL4

over CertiKOS reflects substrate maturity for x86-64 VT-x extensions and seL4’s established Isabelle/HOL proof harness; CertiKOS’s compositional layered approach is an alternative formal substrate for future EXTP ports. Komodo [19] (Ferraiuolo et al., SOSP 2017) verifies a software enclave monitor in Vale verified assembly on ARM. Both show that verifying system software in the virtualization stack is tractable; EXTP builds on seL4 rather than providing a new verified kernel.

The object of mechanization also differs. seL4, CertiKOS, and Komodo machine-check the *substrate* — kernel or monitor correctness, with the proof closing over the implementation. EXTP’s Isabelle development (§VII-C) machine-checks the *analysis method*: CST’s conditional core is proved over an abstract surface and trace model, and substrate correctness enters only as the named hypothesis A_1 . The two are complementary, not nested: a substrate proof discharges A_1 for $\mathcal{K}_{\text{verified}}$ — our development isolates exactly that obligation as one locale assumption — while a verified kernel says nothing about causal attributability of a divergence.

D. Causal Inference in Security

Pearl [1] and Halpern-Pearl [2] provide EXTP’s theoretical foundation. Soloviev, Balliu, and Guanciale [13] (IEEE CSF 2024) introduce a modal logic framework for security properties capturing confidentiality, integrity, and robust declassification variants. This operates at the policy specification level rather than the operational causal attribution level; it complements EXTP’s approach.

Prior applications of causal inference to security take two forms. Dependency tracking approaches such as Back-Tracker [20] (King and Chen, SOSP 2003) follow causal chains through system-call dependencies for intrusion forensics without invoking Pearl’s structural causal models. Probabilistic causality inference approaches such as MCI [21] (Kwon et al., NDSS 2018) construct probabilistic graphical models over audit logs for attack reconstruction. EXTP’s approach is categorically different: causal attribution via controlled counterfactual replay, grounded in a formal soundness theorem, replacing probabilistic assumptions with deterministic trajectory-level guarantees.

IX. CONCLUSION

EXTP shows that sound counterfactual causal attribution in exploit analysis follows from a verified capability substrate (seL4) and three observable properties bounding intervention authority (CapSep), replay determinism (SOE), and timing confounders (MI). The Counterfactual Soundness Theorem proves that their composition grounds Pearl and Halpern-Pearl causal attribution within explicitly enumerated epistemic bounds, and the $A1\text{-modulo}$ discipline surfaces unresolved assumption gaps rather than absorbing them. A bare-metal privilege-check-bypass demonstration exercises the pipeline end-to-end and doubles as an empirical test of CST’s own envelope: naïve and silent intervention hooks produce identical SOE-clause divergence but bifurcate on the timing clause — the framework detecting its own $A1\text{-modulo}$ violations.

CST v1.0’s scope is deliberately narrow (single-VCPU traces, a 4-tuple plus timing surface, per-event intervention scope); its extensions (§VII-A) inherit the threat-elimination structure and require extended empirical antecedents rather than new soundness machinery. The proof’s conditional core is machine-checked in Isabelle/HOL (§VII-C); the seL4 lifting and the calibration/HOL interface remain open, and the conditional theorem does not depend on them. In EXTP’s discipline, the boundary of soundness is named, not avoided.

ETHICS CONSIDERATIONS

All experiments were conducted on isolated bare-metal hardware with no network connectivity. No human subjects, real user data, or production systems were involved. The privilege-check-bypass demonstration uses a controlled synthetic guest modeled on a public exploit pattern (CrackArmor confused-deputy cluster) and does not disclose any new vulnerability. CVE references appearing in §VIII are used only for related work discussion. The counterfactual replay infrastructure operates post-hoc on already- captured exploit traces and provides no novel attack capability.

REFERENCES

- [1] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, UK: Cambridge University Press, 2009.
- [2] J. Y. Halpern and J. Pearl, “Causes and explanations: A structural-model approach. Part I: Causes,” *The British Journal for the Philosophy of Science*, vol. 56, no. 4, pp. 843–887, 2005.
- [3] A. Fariha, S. Nath, and A. Meliou, “Causality-guided adaptive interventional debugging,” in *Proceedings of the 2020 International Conference on Management of Data (SIGMOD)*. ACM, 2020, pp. 431–446.
- [4] M. Soloviev and J. Y. Halpern, “Security properties as nested causal statements,” *CoRR*, vol. abs/2104.00872, 2021, arXiv preprint, 8 pages.
- [5] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, “seL4: Formal verification of an OS kernel,” in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*. Big Sky, MT, USA: ACM, 2009, pp. 207–220.
- [6] G. Klein, J. Andronick, K. Elphinstone, T. Murray, T. Sewell, R. Kolanski, and G. Heiser, “Comprehensive formal verification of an OS microkernel,” *ACM Transactions on Computer Systems*, vol. 32, no. 1, pp. 2:1–2:70, 2014.
- [7] Linux Kernel Security, “CVE-2025-38352: POSIX CPU timer use-after-free race between `handle_posix_cpu_timers()` and `posix_cpu_timer_del()`,” 2025, linux kernel TOCTOU race condition; <https://nvd.nist.gov/vuln/detail/CVE-2025-38352>.
- [8] —, “CVE-2025-68259: KVM SVM INT3 re-injection race with concurrent `text_poke`,” 2025, linux kernel KVM/SVM software-interrupt re-injection bug; upstream fix in commit 4da3768e1820 (6.19-rc1); <https://nvd.nist.gov/vuln/detail/CVE-2025-68259>.
- [9] R. Gu, Z. Shao, H. Chen, X. N. Wu, J. Kim, V. Sjöberg, and D. Costanzo, “CertiKOS: An extensible architecture for building certified concurrent OS kernels,” in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Savannah, GA, USA: USENIX Association, 2016, pp. 653–669.
- [10] R. Zhang, L. Gerlach, D. Weber, L. Hetterich, Y. Lü, A. Kogler, and M. Schwarz, “CacheWarp: Software-based fault injection using selective state reset,” in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. Philadelphia, PA, USA: USENIX Association, 2024, pp. 1135–1151.
- [11] B. Schlüter, S. Sridhara, A. Bertschi, and S. Shinde, “WeSee: Using malicious #VC interrupts to break AMD SEV-SNP,” in *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2024.
- [12] B. Schlüter, S. Sridhara, M. Kuhne, A. Bertschi, and S. Shinde, “Heckler: Breaking confidential VMs with malicious interrupts,” in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. USENIX Association, 2024.
- [13] M. Soloviev, M. Balliu, and R. Guanciale, “Security properties through the lens of modal logic,” in *Proceedings of the 37th IEEE Computer Security Foundations Symposium (CSF)*. IEEE, 2024, pp. 340–355.
- [14] S. Tanda and A. Allievi, “CVE-2024-21305: HVCI bypass via non-secure VMCS configuration,” 2024, mSRC Advisory; technical writeup at <https://tandasat.github.io/blog/2024/01/15/CVE-2024-21305.html>.
- [15] Broadcom, “VMSA-2025-0004: VMware ESXi, Workstation, and Fusion updates address multiple vulnerabilities (CVE-2025-22224, CVE-2025-22225, CVE-2025-22226),” 2025, broadcom Security Advisory, 4 March 2025.
- [16] S. Wiebing, C. Giuffrida *et al.*, “Training solo: On the limitations of domain isolation against spectre-v2 attacks,” in *Proceedings of the 46th IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2025, vUSec, Vrije Universiteit Amsterdam. CVE-2024-28956, CVE-2025-24495.
- [17] ETH Zurich Comsec Group, “VMScape: Exposing and exploiting incomplete branch predictor isolation in cloud environments,” 2025, CVE-2025-40300, September 2025; <https://github.com/comsec-group/vmscape>.
- [18] Xen Project, “XSA-472: Multiple vulnerabilities in the Viridian interface,” 2025, xen Security Advisory, CVE-2025-27466, CVE-2025-58142, CVE-2025-58143; <https://xenbits.xen.org/xsa/advisory-472.html>.
- [19] A. Ferraiuolo, A. Baumann, C. Hawblitzel, and B. Parno, “Komodo: Using verification to disentangle secure-enclave hardware from software,” in *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*. Shanghai, China: ACM, 2017, pp. 287–305.
- [20] S. T. King and P. M. Chen, “Backtracking intrusions,” in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2003, pp. 223–236.
- [21] Y. Kwon, F. Wang, W. Wang, K. H. Lee, W.-C. Lee, S. Ma, X. Zhang, D. Xu, S. Jha, G. F. Ciocarlie, A. Gehani, and V. Yegneswaran, “MCI: Modeling-based causality inference in audit logging for attack investigation,” in *Proceedings of the 25th Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2018.

APPENDIX A

$\mathcal{K}_{\text{EXTENDED}}$ OPERATION ENUMERATION

$\mathcal{K}_{\text{extended}}$ comprises the x86-64 VT-x extension operations through which EXTP’s intervention machinery interacts with guest state outside L4.verified’s formal scope (§II). The full operation set used in CST v1.0:

- `WriteRegisters` — update guest GP register state via `seL4_X86_VCPU_WriteRegisters`. Used by the intervention machinery to apply $\text{do}(X=x')$ for state-class interventions (§IV-C).
- `ReadVMCS / WriteVMCS` — read/write individual VMCS fields. Used by snapshot save/restore (§IV, 23 fields in Appendix C).
- `MapEPT / UnmapEPT` — guest-physical address translation modification. Used by EPT-CoW restore.
- `EPTPermDowngrade / EPTPermRestore` — permission bits manipulation on EPT entries for CoW checkpoint mechanism.
- `VMResume / VMLaunch` — guest re-entry, reading from the (potentially intervention-modified) VMCS and register state.
- `TSCRead (virtualized)` — monotonic TSC seed delivery (§IV); deterministic by construction.

Each operation’s empirical disjointness from $\mathcal{K}_{\text{fwk}}$ is established by the Clopper-Pearson bounds reported in §V-A; within-boot bound $\leq 0.37\%$ at $N=998$, cross-boot bound $\leq 10.4\%$ at $N=34$.

APPENDIX B BUILD PROVENANCE

Toolchain. EXTP VMM compiled with gcc 12.3 at optimization level -O2, no link-time optimization. seL4 microkernel built from the master branch (commit hash recorded at artifact-release time). Build system: CMake + Ninja. Target triple: x86_64-pc99.

Build artifacts.

- kernel-x86_64-pc99 — seL4 microkernel ELF, 3.3 MB.
- extp-vmm-image-x86_64-pc99 — EXTP VMM rootserver ELF, 881 KB.

Build configuration flags for the demonstration run.

EXTP_DEMO_BOOT=ON,
EXTP_NUM_RUNS_OVERRIDE=222,
CST_INTERVENTION_VERBOSE=0 (silent hook). All other EXTP_*_BOOT modes set OFF.

Hardware identity. Intel 12th Gen Core (Alder Lake), microcode revision 0x3e, ASUS Z690 motherboard, UEFI firmware. P-state locked at 4.9 GHz turbo via firmware configuration. No host operating system; seL4 boots directly via the bootloader.

Out-of-band I/O. Serial console: DIGITUS USB-Serial FT232RL, Digitus PCIe Serial DS-30000-1, RS-232 null modem cable at 115200 baud, 8N1.

The Isabelle/HOL development, the measurement summaries behind Figures 2–3 and Table V, the boot-collection protocols, and the two boot images above are available at <https://github.com/GITHUB-USER/extp-artifact>.

APPENDIX C VMCS FIELD ENUMERATION

EXTP’s snapshot/restore mechanism (§IV) preserves 23 VMCS fields covering the guest’s architectural state visible through the VT-x interface, plus four core registers (RIP, RSP, RFLAGS, CR3) handled separately. The 23 VMCS fields fall into three categories:

Guest control state. Guest activity state, interruptibility state, pending debug exceptions, IA32_DEBUGCTL, IA32_SYSENTER_CS / _ESP / _EIP, VMCS link pointer.

Guest segment descriptors. Selector, base, limit, and access-rights fields for the standard guest segments (CS, SS, DS, ES, FS, GS, LDTR, TR), plus GDTR and IDTR base/limit.

Guest exception and entry control. VM-entry-controls, VM-entry-exception-error-code, VM-entry-instruction-length, guest-cr0-read-shadow, guest-cr4-read-shadow.

Non-restored fields (intentionally omitted) include: host-state fields (constant across replay), derived/read-only fields (e.g., exit-information, computed from guest execution), and architectural state covered by the separate RIP/RSP/RFLAGS/CR3 restore path. The exclusion rationale is correctness-preserving: non-restored fields are either invariant under replay or deterministically reconstructed from restored state.

The exhaustive field list with VMCS field encodings and per-field restoration semantics is released in the build manifest (Appendix B).

APPENDIX D FULL PROOFS

This appendix expands the proof sketch of Theorem 1 (§V-D) and provides the inductive proof of Proposition 1. The presentation states each step in standard mathematical notation rather than in Isabelle/HOL syntax, for readability; the conditional core of what follows — the three bounding lemmas, the AC2(b) witness construction, the composition into Theorem 1, and Proposition 1 — has been mechanized separately, with A_1 – A_4 as hypotheses. §VII-C states precisely what the mechanization does and does not establish.

A. Notation and antecedent restatement

Let $\mathbb{E} = (e_0, e_1, \dots, e_n)$ denote a trace of VM-exit events under EXTP’s VMM, with each e_i carrying the 4-tuple observable surface plus per-event timing Δ_i (§IV). Write T_{orig} for the original captured trace and T_{cf} for the counterfactual replay trace produced under intervention $\text{do}(X=x')$. The divergence detection function is

$$D(T_{\text{orig}}, T_{\text{cf}}, i) = [\text{observable}(T_{\text{orig}})_i \neq \text{observable}(T_{\text{cf}})_i] \vee [|\Delta_i^{\text{cf}} - \Delta_i^{\text{orig}}| > \Delta^*].$$

The validity envelope $\mathcal{E}$ is a tuple (instr, wl-class, gw-type, scope, $\mathcal{S}_{\text{in}}$, $\mathcal{S}_{\text{out}}$) as defined in §IV. The four CST antecedents (Table II) restated symbolically:

- **A1 (Modularity).** For any $\text{do}(X=x')$ within the validated intervention surface and any framework-internal observable $W \in \mathcal{K}_{\text{fwk}}$: $\Pr[|W_{\text{cf}} - W_{\text{orig}}| > \Delta^*] \leq \delta_{A1}$ where δ_{A1} is bounded by CapSep’s empirical envelope (§V-A). The current paper asserts A1 *modulo* an unresolved Q4 experiment (the A1-modulo flag).
- **A2 (Observable completeness over $\mathcal{S}_{\text{in}}$).** For interventions in $\mathcal{S}_{\text{in}}$, any causal effect of $\text{do}(X=x')$ on the guest’s execution either manifests as a 4-tuple delta or as a per-event timing deviation above Δ^* . Microarchitectural pathways outside $\mathcal{S}_{\text{in}}$ (§III) are explicitly out of A2’s scope.
- **A3 (Confounder bound).** Per-event timing fluctuations $|\Delta_i| < \Delta^*$ under no intervention, with realized false-positive rate $\alpha_{\text{realized}} \leq \alpha$ (MI §V-C).
- **A4 (Replay determinism).** In the absence of intervention, T_{orig} and any fresh-boot replay R satisfy $\forall i : \text{observable}(T_{\text{orig}})_i = \text{observable}(R)_i$ — the operational statement of SOE (§V-B).

B. Proof of Theorem 1 (atomic CST)

Theorem 1’s biconditional has two directions: the “ $\Rightarrow$ ” direction (admissibility implies the four listed correspondence properties hold) and the “ $\Leftarrow$ ” direction (the four-step admissibility check is sufficient for the claim to be issued).

Direction ($\Leftarrow$): admissibility check $\Rightarrow$ claim is emitted. This direction is procedural: the four-step check (§IV-C) is an executable predicate. By construction, if envelope match,

assumption validation, surface check, and intervention surface predicates all evaluate true, EXTp emits the CST claim; if any evaluates false, no claim is emitted. This direction is established by code-level inspection of the admissibility implementation, not by deductive proof.

Direction ($\Rightarrow$): claim emitted $\Rightarrow$ correspondence properties hold. This is the substantive direction, and we prove it via three lemmas eliminating alternative explanations for the observed divergence.

Lemma 1 (SOE-clause coincidence bound). *Under A4 (SOE replay determinism), the probability that the SOE clause of D fires (4-tuple divergence between T_{orig} and T_{cf} at some event i) in the absence of any trace-distinguishing input is bounded by ε_{SOE} , where ε_{SOE} is the Clopper-Pearson one-sided 95% upper bound on the per-run SOE-clause failure rate established in §V-B: $\varepsilon_{\text{SOE}} \leq 0.37\%$ at $N=998$ (within-boot) and $\varepsilon_{\text{SOE}} \leq 10.4\%$ at $N=34$ (cross-boot).*

Proof. A4 asserts $\forall i : \text{observable}(T_{\text{orig}})_i = \text{observable}(R)_i$ across the calibrated trial set, with realized failure count zero. The Clopper-Pearson construction translates the zero-failure observation at sample size N into a one-sided upper confidence bound on the true per-run failure rate. Under the single-configuration discipline, the only controlled trace-distinguishing input between T_{orig} and T_{cf} is $\text{do}(X=x')$; any other source of divergence must therefore arise from the residual failure mass bounded above by ε_{SOE} . The SOE-clause coincidence probability is thus bounded by ε_{SOE} , and the bound is reported per-claim via the validity envelope $\mathcal{E}$. $\square$

Lemma 2 (Timing-clause coincidence bound). *Under A3 (MI per-event timing threshold), the probability that the timing clause of D fires ($|\Delta_i^{\text{cf}} - \Delta_i^{\text{orig}}| > \Delta^*$) at any single event i in the absence of intervention is bounded by α_{realized} , the empirical false-positive rate calibrated per workload class in §V-C.*

Proof. A3’s empirical content is the calibrated per-event FPR: $\Pr[|\Delta_i^{\text{cf}} - \Delta_i^{\text{orig}}| > \Delta^* \mid \text{no intervention}] \leq \alpha_{\text{realized}}$, with $\alpha_{\text{realized}} = 0.61\%$ under L1-pchase/ S_1 (boot_log8, $N=978$) and 2.78% under L1-pchase/ S_1 at finer grain (boot_log12, $N=108$) — both bounded below the nominal $\alpha = 5\%$ by the conservative Gaussian threshold construction (§V-C). The bound is per-event; trace-level aggregation is the downstream consumer’s responsibility (§V-C, “Scope: per-event detection”). The timing-clause coincidence probability at any single event i is thus bounded by α_{realized} for the calibrated workload class. $\square$

Remark on naming. The previous draft labeled these as *elimination* lemmas; we adopt *bounding* to reflect the probabilistic structure of the result. Coincidence is not eliminated — it is bounded above by empirically calibrated envelopes (ε_{SOE} for the structural clause, α_{realized} for the timing clause). Both bounds are propagated into the validity envelope $\mathcal{E}$ of every CST claim, making coincidence a *reported* rather than *assumed-away* source of uncertainty.

Lemma 3 (Scope-honest restriction over $\mathcal{S}_{\text{in}}$). *Under A2 restricted to $\mathcal{S}_{\text{in}}$, EXTp’s emission policy guarantees that no admissible CST claim is issued without observable divergence on T_{orig} ’s surface. Pathways in $\mathcal{S}_{\text{out}}$ fall outside A2’s coverage and are excluded by construction.*

Remark on lemma content. The substantive claim of Lemma 3 is not that A2 implies its own scope (which would be definitional), but that EXTp’s emission policy — an independent design choice of the framework, not part of A2’s content — binds itself precisely to A2’s surface. The lemma states the consistency of these two independently specified components: A2 specifies *what is observable*, while the emission policy specifies *under what observation claims are issued*.

Proof. Suppose $\text{do}(X=x')$ causally produces an effect on the guest’s execution within $\mathcal{S}_{\text{in}}$. By A2’s definition of $\mathcal{S}_{\text{in}}$ (§V-D antecedents), this effect manifests as either a 4-tuple delta or a timing deviation above Δ^* , i.e., $D(T_{\text{orig}}, T_{\text{cf}}, i) = \text{true}$ for some event i . The emission policy (§IV-C, step 3) issues a CST claim if and only if $D = \text{true}$ at some event i . Therefore:

- If $D = \text{true}$: the divergence is on T_{orig} ’s observable surface (by A2’s $\mathcal{S}_{\text{in}}$ definition), and the emitted claim is an attribution at the surface level.
- If $D = \text{false}$ at all i : no claim is emitted (by the emission policy). CST is scope-honestly silent.

The lemma’s content is the *binding* between A2’s observable surface and the emission policy’s trigger condition. Pathways in $\mathcal{S}_{\text{out}}$ — silent microarchitectural channels (§III) — produce no signal on D and therefore no claim, but this is not a soundness statement about $\mathcal{S}_{\text{out}}$ (CST does not claim correctness for $\mathcal{S}_{\text{out}}$); it is a scope statement that $\mathcal{S}_{\text{out}}$ is excluded from CST’s attribution surface by construction.

Two consequences of this lemma are worth separating, because they assign A2 a role distinct from that of the other antecedents. The first is the direction just proved — CST does not speak without observable divergence — which follows from the emission policy alone. The second is its converse over $\mathcal{S}_{\text{in}}$: under A2, an in-scope causal effect *cannot* go unreported, so CST’s silence is informative about $\mathcal{S}_{\text{in}}$ (and, as the scope statement above records, about nothing else). A1, A3, and A4 underwrite the *soundness* of claims that are issued; A2 underwrites the *scope-completeness* of the claims that are not. This is why A2 does not appear in the derivation of the correspondence properties (i)–(iv) above: its work is done on the other side of the emission boundary. Both directions are machine-checked (§VII-C).

We note also that A2 is a falsifiable assumption rather than a restatement of D : a perturbation can be real and yet leave no trace on either clause — a timing shift strictly below Δ^* is the elementary case — and were such a perturbation an in-scope causal effect, A2 would be violated. This is the sub- Δ^* silent-leak class of §V-C viewed from the assumption side, and it yields a sharp scope boundary: the mechanized development proves that under A2 an in-scope silent leak is impossible, so *every* silent leak necessarily lies in $\mathcal{S}_{\text{out}}$ (§VII-C). The

deferral of the silent-leak class to a future CMD property class is thus not a matter of convention but a consequence of the assumption structure. The CMD property class (§VII) extends A2’s coverage into $\mathcal{S}_{\text{out}}$ via PMU-counter surfaces; the emission policy’s structure does not change, only the surface it binds to. $\square$

Lemma 4 (Confounder bounding modulo A1). *Under A1 (CapSep modularity, §V-A), the probability that a framework-internal confounder $Z \in \mathcal{K}_{\text{fwk}}$ — correlated with $\text{do}(X=x')$ and influencing observable(X) $_i$ — accounts for the observed divergence D is bounded above by $\delta_{A1} + \alpha_{\text{realized}}$, where δ_{A1} is CapSep’s Clopper-Pearson empirical envelope and α_{realized} is MI’s calibrated per-event FPR (§V-C).*

Proof. We bound the contribution of framework-internal confounders by case analysis on the magnitude of their manifestation on framework observables.

Pearl’s modularity axiom (realized by CapSep as $\mathcal{K}_{\text{int}} \cap \mathcal{K}_{\text{fwk}} = \emptyset$) requires that $\text{do}(X=x')$ severs all incoming causal connections to X that pass through framework-internal pathways. The empirical content of A1 is the probabilistic statement: for any $W \in \mathcal{K}_{\text{fwk}}$, $\Pr[|W_{\text{cf}} - W_{\text{orig}}| > \Delta^*] \leq \delta_{A1}$, where δ_{A1} is bounded by CapSep’s Clopper-Pearson envelope on $\mathcal{K}_{\text{extended}}$ operations (§V-A; $\delta_{A1} \leq 0.37\%$ within-boot at $N=998$, $\leq 10.4\%$ cross-boot at $N=34$). For $\mathcal{K}_{\text{verified}}$ operations, $\delta_{A1} = 0$ by inheritance from L4.verified.

Suppose a confounder $Z \in \mathcal{K}_{\text{fwk}}$ exists that correlates with $\text{do}(X=x')$ and influences observable(X) $_i$. Z ’s contribution decomposes into two regimes:

Case 1: Z manifests as a framework-observable side-effect above Δ^ .* By A1, this regime occurs with probability $\leq \delta_{A1}$ per framework observable. Such side-effects are detected by CapSep’s empirical surface (§V-A); when detected, they appear in the validity envelope as a CapSep-flagged anomaly and the affected claim is either tagged or withheld. The naïve-hook counter-demonstration (§VI-B) is an empirical instance: the intervention machinery’s `printf`-induced side-effect exceeded Δ^* at +26.6M cycles $\approx 2,595 \cdot \Delta^*$, was correctly detected, and the framework reported the A1-modulo flag.

Case 2: Z manifests as a side-effect below Δ^ .* MI’s $\hat{\sigma}_{\text{baseline}}$ calibration (§V-C) was performed under the silent-hook intervention implementation (§VI-B), with the intervention machinery active during baseline sampling. Framework-observable side-effects produced by the silent-hook apparatus are therefore included in $\hat{\sigma}_{\text{baseline}}$ by construction, not added as a confounder on top of it. By A3, sub- Δ^* side-effects are statistically indistinguishable from baseline timing noise under this calibration discipline; their contribution to coincidence-clause firing is bounded by α_{realized} .

Summing the two regimes by union bound, the total confounder contribution is bounded above by $\delta_{A1} + \alpha_{\text{realized}}$. Both terms are reported per-claim in the validity envelope $\mathcal{E}$. The A1-modulo flag propagates to every CST claim issued under this paper’s calibration: the bound is *named*, not *eliminated*, and the Q4 modularity experiment (§VII) targets strict-form re-

covery ($\delta_{A1} \rightarrow 0$) by directly validating side-effect freedom on $\mathcal{K}_{\text{fwk}}$ observables across all validated intervention classes. $\square$

Remark on naming. The previous draft labeled this an *elimination* lemma; we adopt *bounding modulo A1* to reflect the probabilistic-empirical structure. No formal contradiction is reached against confounder existence; instead, the confounder contribution to attribution uncertainty is bounded above by an explicitly calibrated envelope and propagated, in keeping with CST’s broader epistemic discipline (§V-A, A1-modulo flag).

Composition into Theorem 1. Given the three lemmas, the four correspondence properties (i)–(iv) in Theorem 1 follow directly:

(i) *Effectiveness.* By construction of the intervention machinery (§IV-C), $\text{do}(X=x')$ sets X to x' at event e_i ; this is mechanically verifiable from the post-intervention VMCS state.

(ii) *Composition (δ_{A1} -relaxed form).* Pearl’s composition axiom in its strict form requires that intervening on X while holding W at its natural value w produces the same outcome as $\text{do}(X=x')$ alone when $W_x = w$ — an *exact* equivalence between two structural assignments. EXTP’s empirical bounding of A1 yields a relaxed form: by Lemma 4, W ’s framework-internal value W_{cf} satisfies $|W_{\text{cf}} - W_{\text{orig}}| < \Delta^*$ with probability at least $1 - \delta_{A1}$, rather than $W_{\text{cf}} = W_{\text{orig}}$ exactly.

The δ_{A1} -relaxed composition axiom is the statement: for all $W \in \mathcal{K}_{\text{fwk}}$, the outcome under $\text{do}(X=x')$ equals the outcome under $\text{do}(X=x')$ while holding $W = w_{\text{orig}}$, up to a perturbation bounded by Δ^* on each W with probability at least $1 - \delta_{A1}$.

Two observations on this relaxation:

- *Recovery of strict form.* In the limit $\delta_{A1} \rightarrow 0$, the relaxed form reduces to Pearl’s strict composition. The Q4 modularity experiment (§VII) targets this limit by validating strict side-effect freedom on $\mathcal{K}_{\text{fwk}}$ observables. Both the relaxed form (holding $W \subseteq \mathcal{K}_{\text{fwk}}$ at its T_{orig} values, per-variable perturbation $\leq \Delta^*$, per-variable violation mass $\leq \delta_{A1}$) and its strict-form recovery are machine-checked (§VII-C).
- *Asymmetry vs. faithfulness.* Where CST satisfies a *stronger* form of faithfulness (subsumed by SOE’s trajectory determinism, §V-D), it satisfies a *relaxed* form of composition. This asymmetry is unavoidable: composition operates on framework-internal state $W \in \mathcal{K}_{\text{fwk}}$, governed by $\mathcal{K}_{\text{extended}}$ operations whose disjointness from $\mathcal{K}_{\text{int}}$ is empirically (not structurally) bounded; faithfulness operates on the observable surface, where SOE provides structural-grade determinism. The relaxation is therefore not a methodological choice but a direct consequence of the verified/extended boundary in seL4’s capability model (§II).

(iii) *HP AC1.* D occurs in T_{cf} at event i by the third admissibility step ($D = \text{true}$); AC1’s “actual world” is T_{cf} in CST’s reframing (Theorem 1 footnote).

(iv) *HP AC2(a) and AC2(b).* We treat the two sub-conditions separately.

AC2(a): counterfactual sensitivity. AC2(a) requires that under an alternative assignment to the cause, the effect changes.

By construction of D , when $D(T_{\text{orig}}, T_{\text{cf}}, i) = \text{true}$, the $\text{observable}(\cdot)_i$ at event i differs between T_{orig} (under natural assignment $X = x$) and T_{cf} (under $\text{do}(X=x')$). The structural-clause version (4-tuple delta) and timing-clause version ($|\Delta_i^{\text{cf}} - \Delta_i^{\text{orig}}| > \Delta^*$) each independently witness AC2(a); independence here is not merely stipulated — neither clause entails the other, as witnessed by traces on which exactly one fires. The third admissibility step ($D = \text{true}$) is therefore exactly the operational test of AC2(a).

AC2(b): witness fixity. AC2(b) requires that the AC2(a) sensitivity hold while a specified witness set W of variables is fixed at their actual values from T_{orig} . In Halpern-Pearl’s formulation, W is the set of variables outside the cause set that, if allowed to take their natural counterfactual values, might mask the causal effect — the “demoting” set. The construction of W is non-trivial: in classical HP, W must be chosen such that (i) fixing $W = w_{\text{orig}}$ does not itself force the effect to change, and (ii) under $\text{do}(X=x') \wedge (W = w_{\text{orig}})$, the effect still differs from T_{orig} .

EXTp’s correspondence to AC2(b) proceeds by constructive identification of W with the partition $(\mathcal{S}_{\text{in}}, \mathcal{S}_{\text{out}})$ defined in the validity envelope (§IV):

Construction. Partition the full execution state space at event i into three disjoint sets:

- $\{X\}$: the intervention target (cause variable).
- $\mathcal{S}_{\text{in}}$: framework-observable variables within the validated intervention surface, distinct from X . Per §IV-C, $\mathcal{S}_{\text{in}}$ is enumerated as part of the validity envelope and includes the observable 4-tuple components (RIP, exit_reason, exit_qual, RAX) plus timing.
- $\mathcal{S}_{\text{out}}$: variables outside the validated intervention surface — silent microarchitectural channels (cache, branch predictor, TLB), non-RAX GPRs, framework-internal scheduler state.

We identify the HP witness set $W = \mathcal{S}_{\text{in}} \setminus \{\text{observable}(\cdot)_i\}$: framework-observable variables that EXTp’s apparatus can both *read* (capture in T_{orig}) and *hold fixed* across replays, excluding the variables that constitute the effect itself.

Verification of witness conditions. Two properties must hold for W to serve as an HP witness set under AC2(b):

(W1) Fixity is operationally enforceable. For each $w \in W$, the value w_{orig} observed in T_{orig} is reproduced exactly in T_{cf} in the absence of intervention. This is precisely A4 (replay determinism, §V-B): score_S = 1.0000 over $N=34$ cross-boot trials establishes that all observables in $\mathcal{S}_{\text{in}}$ (which contains W) are reproduced trajectory-equivalent under fresh-boot discipline, up to the Clopper-Pearson envelope $\varepsilon_{\text{SOE}} \leq 10.4\%$ per-run. Fixity is therefore enforceable up to A4’s empirical bound, not by additional operational machinery beyond fresh-boot replay.

(W2) Fixity does not mask the effect. When $W = w_{\text{orig}}$ is enforced via fresh-boot determinism, the AC2(a) sensitivity must still hold — i.e., the divergence D between T_{orig} and T_{cf} must arise from $\text{do}(X=x')$, not from some $w \in W$ taking a different value. This is established by Lemma 1 (SOE-clause coincidence bound) combined with Lemma 4 (confounder bounding modulo A1):

- Lemma 1 bounds the probability that $\text{observable}(\cdot)_i$ differs between T_{orig} and a no-intervention replay R by ε_{SOE} ; under fresh-boot determinism, $W = w_{\text{orig}}$ holds with probability $\geq 1 - \varepsilon_{\text{SOE}}$.
- Lemma 4 bounds the probability that any framework-internal confounder $Z \in \mathcal{K}_{\text{fwk}}$ produces the observed divergence by $\delta_{A1} + \alpha_{\text{realized}}$.

Together: the probability that $D(T_{\text{orig}}, T_{\text{cf}}, i) = \text{true}$ arises from a W -violation rather than from $\text{do}(X=x')$ is bounded above by $\varepsilon_{\text{SOE}} + \delta_{A1} + \alpha_{\text{realized}}$, the same envelope that propagates into the validity envelope $\mathcal{E}$ of every CST claim.

Conclusion. $W = \mathcal{S}_{\text{in}} \setminus \{\text{observable}(\cdot)_i\}$ is an HP witness set under AC2(b) by construction, with (W1) (fixity enforceability) discharged by A4 and (W2) (fixity does not mask) discharged by Lemmas 1 and 4 jointly. The witness-fixity claim is therefore *derived* rather than asserted, and is bounded by the same empirical envelope that governs the rest of CST.

Witness-refined epistemic envelopes. The envelope $\varepsilon_{\text{SOE}} + \delta_{A1} + \alpha_{\text{realized}}$ derived above is a union bound taken over *both* clauses of D . But AC2(a) is witnessed by one specific clause, and the two clauses do not carry the same coincidence mass: the structural clause is bounded by ε_{SOE} (Lemma 1) while the timing clause is bounded by α_{realized} (Lemma 2). Conditioning on the witness therefore yields two distinct envelopes, each adding the confounder term $\delta_{A1} + \alpha_{\text{realized}}$ of Lemma 4:

structural witness: $\varepsilon_{\text{SOE}} + \delta_{A1} + \alpha_{\text{realized}}$,

timing witness: $\delta_{A1} + 2\alpha_{\text{realized}}$.

Comparing the two reduces, after cancellation, to a single inequality: the structural witness gives the tighter envelope *iff* $\varepsilon_{\text{SOE}} \leq \alpha_{\text{realized}}$. This criterion is not decided once and for all by the framework — it is decided by the calibration regime, and the two regimes reported in §V-A fall on opposite sides of it. Within-boot ($\varepsilon_{\text{SOE}} \leq 0.37\%$ at $N=998$, $\alpha_{\text{realized}} = 0.61\%$) the structural witness is tighter: 1.35% against 1.59%. Cross-boot ($\varepsilon_{\text{SOE}} \leq 10.4\%$ at $N=34$) the ordering reverses and the timing witness is tighter: 11.62% against 21.41%. The practical consequence is that a consumer of CST claims should report the envelope of the witness that actually fired rather than the blanket union bound, and that the question “which clause carries more evidential weight?” has no regime-independent answer. Both envelopes and the comparison criterion are machine-checked (§VII-C).

Remark on minimality (AC3 deferred). HP’s AC3 minimality condition requires that no proper subset of the cause set is sufficient. CST v1.0 takes the cause set to be the singleton $\{X\}$ for atomic interventions; AC3 minimality is trivially satisfied for singletons. For sequential composition (Proposition 1), AC3 is non-trivial and deferred to future work (§VII-A).

Remark on $\mathcal{S}_{\text{out}}$. Variables in $\mathcal{S}_{\text{out}}$ — silent microarchitectural channels and unobserved GPRs — lie *outside* the witness set W by construction. AC2(b) does not require fixity over $\mathcal{S}_{\text{out}}$; HP only requires fixity over a chosen witness set, not over all non-cause variables. The scope-honesty discipline (§V-D, Threat 2; Lemma 3) ensures that CST issues no claim

when divergence manifests exclusively through S_{out} variables ($D = \text{false}$ on the observable surface), since the resulting attribution would lack AC2(a) sensitivity altogether.

Thus all four properties hold under the assumption envelope (A_1, A_2, A_3, A_4) with the explicit qualifiers documented in Table III: composition in δ_{A1} -relaxed form, AC2(b) derived rather than asserted, and the residual epistemic envelope $\varepsilon_{\text{SOE}} + \delta_{A1} + \alpha_{\text{realized}}$ reported per-claim. $\square$

C. Proof of Proposition 1 (sequential composition)

Proof of Proposition 1. By induction on chain length n .

Base case ($n=1$). The chain $\langle \text{do}(X_1) \rightsquigarrow_{i_1} D \rangle$ is a single atomic claim, admissible iff Theorem 1 holds, which is given.

Inductive step. Assume admissibility of the n -step chain $\langle \text{do}(X_1) \rightsquigarrow_{i_1} \dots \rightsquigarrow_{i_n} D \mid A; \mathcal{E} \rangle$. We show admissibility of the $(n+1)$ -step extension $\langle \text{do}(X_1) \rightsquigarrow_{i_1} \dots \rightsquigarrow_{i_n} \rightsquigarrow_{i_{n+1}} D' \mid A; \mathcal{E} \rangle$, where D' is the divergence observed after the $(n+1)$ -th intervention.

By the inductive hypothesis, the prefix chain produces a $T_{\text{cf}}^{(n)}$ trace whose observable surface and timing satisfy the four correspondence properties of Theorem 1 under $(A_1^{(n)}, A_2, A_3, A_4)$, where $A_1^{(n)}$ denotes the A1 bound after n interventions. The $(n+1)$ -th intervention, $\text{do}(X_{n+1} = x'_{n+1})$, is by construction a member of the validated intervention surface and operates on $T_{\text{cf}}^{(n)}$ as its baseline. Applying Theorem 1 to the atomic step $\langle \text{do}(X_{n+1}) \rightsquigarrow_{i_{n+1}} D' \mid A; \mathcal{E} \rangle$ with $T_{\text{cf}}^{(n)}$ as the original trace yields admissibility for the extended chain, with $A_1^{(n+1)} \leq A_1^{(n)} + \delta_{A1}$ (A1's modulo bound accumulating by union bound across interventions).

The key non-trivial step is that CapSep's per-operation disjointness (§V-A) is order-independent: each $\mathcal{K}_{\text{int}}$ operation remains disjoint from $\mathcal{K}_{\text{fwk}}$ regardless of execution order, so A1's structural component does not degrade under sequential composition. The empirical δ_{A1} accumulates by union bound; tighter bounds under independence are not asserted. The accumulated envelope is reported in $\mathcal{E}$.

Commutativity is *not* asserted: the order $X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_{n+1}$ is part of the claim's specification. Reordering to a permutation $X_{\pi(1)} \rightarrow \dots \rightarrow X_{\pi(n+1)}$ produces a distinct claim that must be verified separately. The induction therefore proves admissibility under the specified ordering only. The chain, its one-step extension with $T_{\text{cf}}^{(n)}$ as the new baseline, and the per-step application of Theorem 1 are machine-checked (§VII-C). $\square$

Scope clarification: sequential vs. parallel ensembles. Proposition 1 governs *sequential* composition — chains of interventions applied within the same T_{cf} execution, where $\text{do}(X_{k+1})$ operates on the state produced by $\text{do}(X_k)$. Parallel ensembles of atomic interventions — sets of distinct T_{cf} runs, each applying a single $\text{do}(X_i)$ from independent fresh-boot state — fall under Theorem 1 applied per-instance and are not subject to the union-bound accumulation of δ_{A1} across i . The counterfactual sweep of §VI-B5 is an instance of the latter: 33 independent atomic claims, each verified against T_{orig} independently, with δ_{A1} bounded per-claim by CapSep's

empirical envelope. Sweep coverage scaling is therefore $O(n)$ in the number of intervention points without compositional bound degradation.

For a sweep whose claims one wishes to aggregate into a single ensemble-level envelope, the witness refinement above sharpens the naive figure. A claim witnessed structurally carries $\varepsilon_{\text{SOE}} + \delta_{A1} + \alpha_{\text{realized}}$; one witnessed by timing carries $\delta_{A1} + 2\alpha_{\text{realized}}$. If k of n claims fire structurally, the aggregate envelope is $k(\varepsilon_{\text{SOE}} + \delta_{A1} + \alpha_{\text{realized}}) + (n-k)(\delta_{A1} + 2\alpha_{\text{realized}})$, which is bounded above by n times the per-claim maximum and meets that naive bound only when every claim fires on the same worst-case clause. A sweep that mixes witnesses is therefore strictly tighter: for two claims of opposite witness under the within-boot calibration, the aggregate is 2.94% against the naive 3.18%, and the gap grows linearly in the number of mixed-witness claims. Applied to the sweep of §VI-B5 — one structural claim among 32 silent inputs — the aggregate envelope is that of a single structural claim regardless of the 32 silent inputs, since silent inputs emit no claim. This ensemble bound, its strictness, and the sweep instantiation are machine-checked (§VII-C).

D. Scope and limitations of these proofs

These proofs establish CST as a mathematical theorem under EXTp's notation and antecedent system. Three explicit limitations apply.

Mechanized fragment. Table VI maps the components of the argument above to the corresponding facts in the Isabelle/HOL development (115 facts, no unproved obligations, Isabelle2025-1). Rows marked *assumed* are hypotheses of the `cst_assumptions` or `sel4_lifting` locale, not theorems — in particular the three bounding lemmas are stated in Isabelle as the operational content of A_1, A_3, A_4 (the locale carries them as axioms and the development propagates them), not derived from a calibration model; rows marked *model* are consistency witnesses showing the locales are non-vacuous.

Residual mechanization gap. The trace data type, the divergence predicate D , the validity envelope, the four-step admissibility check, and the lemmas above have been formalized in Isabelle/HOL and are machine-checked with A_1 – A_4 as hypotheses (§VII-C). What remains unmechanized is the interface at either end: the lifting to seL4's existing machine model, which would let A_1 be discharged for $\mathcal{K}_{\text{verified}}$ rather than assumed, and the representation of the empirical antecedents as structural predicates. The lifting side is scaffolded as a typed interface: the mechanization defines the abstract-state carrier and the $\mathcal{K}_{\text{verified}}/\mathcal{K}_{\text{extended}}$ partition, isolates the single external-atomicity obligation `L4.verified` would supply as one named assumption, and proves that *under it* $\mathcal{K}_{\text{verified}}$ modularity holds unconditionally ($\delta_{A1}=0$) while $\mathcal{K}_{\text{extended}}$ remains outside its scope and thus empirical — but the obligation itself is assumed, not discharged against seL4's proof, which is the multi-year step. The prose above is therefore the human-readable presentation of a checked conditional argument, not a substitute for one; its antecedents are assumed, not proved, and that is where the epistemic weight sits.

TABLE VI
CORRESPONDENCE BETWEEN THE PROOF OF APPENDIX D AND THE ISABELLE/HOL DEVELOPMENT.

Paper component	Isabelle fact	Status
A_1 – A_4 (Table II)	locale cst_assumptions	assumed
Effectiveness of $\text{do}(X=x')$	apply_iv_severs, effective_holds	proved
Pearl composition (δ_{A1} -relaxed, $W/\mathcal{K}_{\text{fwk}}$ form)	composition_W_holds, composition_strict_recovered	proved
$\Leftarrow$ direction (emission)	left_direction	proved
Lemma 1	coincidence_soe_bound	assumed (A-content)
Lemma 2	coincidence_timing_bound	assumed (A-content)
Lemma 4	confounder_bound	assumed (A-content)
Lemma 3 (effect $\Rightarrow D$; effect $\Rightarrow \text{emit}$)	silent_no_missed_effect, silent_emission_complete	proved
HP AC2(a), two witnesses	ac2a_iff_wit, witnesses_logically_independent	proved
HP AC2(b) + union bound	ac2b_holds, w2_residual_bound	proved
Per-witness envelope, regime criterion	structural_bound_tighter_iff	proved
Theorem 1 ($\Rightarrow$)	cst_conditional	proved
Scope-completeness (A2)	cst_scope_complete, silent_leak_only_out_of_scope	proved
Proposition 1, sequential (trace-threaded)	chain_ok_iff_steps, chain_extend, chain_conditional	proved
Proposition 1, parallel ensemble	sweep_three_level_bound	proved
Sweep instantiation (§VI-B)	dead_sweep_envelope_vs_naive_swept	proved
Within-/cross-boot calibration	cst_withinboot, cst_crossboot	model
$\mathcal{K}_{\text{verified}}/\mathcal{K}_{\text{extended}}$ partition	k_partition_disjoint, k_partition_complete	proved
seL4 external atomicity	verified_no_fwkw_touch	assumed
A_1 for $\mathcal{K}_{\text{verified}}$ ($\delta_{A1}=0$)	A1_unconditional_for_verified	proved (under above)

Empirical content of A1, A3, A4. A1’s δ_{A1} , A3’s α_{realized} , and A4’s high-confidence empirical score_S are not derived from first principles; they are estimated from calibration data (§§V-A, V-C, V-B). The theorem is conditional on these empirical antecedents; the proof operates on the antecedents symbolically.

Out-of-scope mechanism classes. The proofs do not address multi-VCPU race conditions (out of A4’s deterministic-replay scope) or silent microarchitectural channels (out of A2’s S_{in} scope). These are explicit scope boundaries (§III), not theorem failures: CST issues no claim under conditions where its antecedents do not hold.

APPENDIX E USE OF AI TOOLS

In accordance with the NDSS 2027 generative AI disclosure policy, the authors disclose the following.

Anthropic’s Claude was used during paper preparation in an assistive capacity, limited to: (i) iterative refinement of figure-rendering code (matplotlib scripts for Figures 2–4 and TikZ code for Figure 1); (ii) editorial polishing of phrasing in selected paragraphs and figure captions, comparable in scope to copy-editing for grammar, clarity, and notation consistency; (iii) sanity-checking of mathematical notation across sections (e.g., consistent use of macros such as T_{orig} , T_{cf} , Δ^*).

The scientific contributions of this paper — the formulation of the Counterfactual Soundness Theorem, the three-property decomposition (CapSep, SOE, MI), the A1-modulo flag propagation discipline, the Pearl/Halpern-Pearl correspondence mapping, the threat model and validity envelope construction, the experimental design and execution (cross-boot trials, $\hat{\sigma}_{\text{baseline}}$ calibration, intervention sweep), all data collection, and the proofs in Appendix D — are the authors’ original intellectual work. No section, subsection, theorem, definition, proof step, claim, or empirical result was generated

by AI tools. The authors take full responsibility for the veracity and correctness of all material in this paper.